

基于差分进化算法的烟幕干扰弹投放策略研究

摘要

烟幕干扰弹主要通过化学燃烧或爆炸分散在目标前方特定空域形成遮蔽。本文针对不同无人机数量，不同速度大小与方向以及多个烟幕弹和多个导弹等参数，建立起以最小化干扰时长为目标的烟幕干扰弹投放规划模型，并以**变步长搜索算法**，**差分进化算法**等算法对该全局搜索空间广泛的烟幕弹规划模型进行求解，最后求得无人机，烟幕弹等优化参数的具体安排策略，使得对导弹的干扰噪声大，对真目标有效遮蔽时间尽可能长。

针对问题一，本文于单个无人机使用单个烟幕弹对单个导弹进行干扰，建立起单烟幕弹单投放定值模型。首先以经典动力学方程刻画无人机、导弹和烟幕弹随时间变化的运动轨迹，以**广义圆锥方程**与**点定法**判断真目标圆柱体所在的相对位置，然后以时间为变步长，其次利用**变步长搜索算法**，先粗后细，带入题中所给数值，对第一问进行求解，最后求出干扰时长为 1.3720s，并且以各种函数的连续性证明来验证变步长搜索算法使用的可行性。

针对问题二，本文于单无人机单烟幕弹对单个导弹进行干扰，建立起以最大化干扰时长为目标的数学规划模型。首先任然以经典动力学方程刻画无人机、导弹等运动轨迹，以**圆锥方程**和**点定法**判断真目标较圆锥的内外位置，其次对于该多参数变量的数学规划模型，本文类比问题一，采用先粗搜后细调的网格搜索算法与 **BFGS 算法**，对该模型进行求解，最后求出最长有效遮蔽时长为 4.5873s。

针对问题三，本文于单无人机三烟幕弹对单个导弹进行干扰，建立起相应的规划模型，对单个无人机三次投放的过程以一条时间链进行刻画，同时其余模型与前两问的模型相同，接着利用**差分进化算法**，逐步迭代求解模型然后进一步讨论烟幕弹与烟幕弹单个，组合，重叠等情况，最后得出在第三问干扰时长结果分别为 **3.5379s**，**2.6125s**，**1.4246s**，详细见 result1.xlsx。

针对问题四，本文于三无人机单烟幕弹对单个导弹进行干扰，任然建立起统筹优化的规划模型，本文首先以**贪心思想**，利用问题二的算法程序分别对三台无人机进行求解，在预求解过程得知三台无人机的**最优解差距较远**，则验证无人机工作的**独立性**，满足贪心算法求得全局最优的问题特征，欲以多阶段局部优解求到全局优解，最后求得干扰时长优解为 **11.40s**，详细见 result2.xlsx。

针对问题五，本文于五无人机三烟幕弹对三个导弹进行干扰，建立起相应的多目标规划模型，由于整体模型的复杂性，本文在综合考虑烟幕弹的个体，组合，重叠等多种因素的基础上，对圆柱体位置判断进行简化，然后在简化模型的基础上，利用差分进化算法进行求解，在迭代足够多的次数，求出目标函数的最大有效遮蔽分数为 **44.0**，详细见 result3.xlsx。最后以**灵敏度分析**对差分进化算法进行验证。

本文特色在于，以广义圆锥方程与点定法，确定真目标圆柱体所在的相对位置，其次以差分进化算法等优化算法，既高效也准确求出复杂烟幕弹的多种混合情况下的有效干扰解，对实际现实具有一定的意义。

关键字： 差分进化算法 变步长搜索法 数学规划 烟幕干扰 无人机投放

一、问题重述

1.1 问题背景

烟幕干扰弹的基本原理是通过燃烧或爆炸形成烟幕云团，在目标前方空域形成遮蔽，以干扰来袭武器。在实战中，无人机可挂载烟幕弹，根据任务需求完成定点精确抛撒。研究烟幕弹的投放策略不仅具有重要的军事应用价值，也对数学模型的建立提出了复杂而深刻的要求。如何通过数学模型精确描述无人机的飞行、烟幕弹的下落、烟幕云团的运动以及与高速导弹之间的时空关系，都是需解决的重要内容。

本次研究的主要目标是通过数学建模手段，对烟幕弹的投放过程进行深入分析，以解决烟幕干扰中涉及的多个实际问题，包括确定无人机的飞行方向与速度、规划烟幕弹的投放点与起爆点等。这不仅为提升无人机烟幕干扰的作战效能提供了理论支持，也为数学建模在现代防御系统中的应用开拓了新的领域。^[1]

1.2 问题提出

本文主要研究无人机投放烟幕干扰弹的策略问题。题设给定了来袭导弹、我方无人机的初始状态，以及烟幕云团的运动与遮蔽机理。

问题一：若无人机 FY1 以 120 m/s 的速度朝向假目标方向飞行，受领任务 1.5 s 后投放 1 枚烟幕干扰弹，该弹间隔 3.6 s 后起爆，计算该烟幕弹对 M1 的有效遮蔽时长。

问题二：利用无人机 FY1 投放 1 枚烟幕干扰弹实施对 M1 的干扰，确定 FY1 的飞行方向、飞行速度、烟幕干扰弹投放点、烟幕干扰弹起爆点，使得遮蔽时间尽可能长。

问题三：利用无人机 FY1 投放 3 枚烟幕干扰弹，实施对 M1 的干扰，给出最优的投放策略。

问题四：利用 FY1、FY2、FY3 等 3 架无人机，各投放 1 枚烟幕干扰弹，实施对 M1 的干扰，给出最优的投放策略。

问题五：利用 5 架无人机，每架无人机至多投放 3 枚烟幕干扰弹，实施对 M1、M2、M3 等 3 枚来袭导弹的干扰，给出最优的投放策略。

二、问题分析

问题一要求在给定无人机固定飞行策略下，计算单枚烟幕弹对导弹 M1 的总有效遮蔽时长。可以首先分别建立无人机、导弹和烟幕弹的运动学方程。其次，建立一个基于遮挡视锥的几何模型，用以判定任意时刻的遮蔽状态。最后，通过编写程序，对整个过程进行模拟，累加所有满足遮蔽条件的步长，得到总有效遮蔽时长。

问题二要求寻找一组最优的无人机飞行与投放策略，使单枚烟幕弹的有效遮蔽时长最大化。该问题是一个以遮蔽时长为目标的非线性优化问题，决策变量为无人机的速度、航向、投放时间及引信延迟。可对可行解域进行几何约束。随后，再混合算法进行求解，即先全局粗筛可行解，再进行局部精炼，最终得到最优策略。

问题三，在用无人机 FY1 投放 3 枚烟幕干扰弹，实施对 M1 的干扰，要求建立烟幕干扰弹的单无人机三枚烟幕干扰弹模型，并且需要我们在综合考虑 3 枚烟幕干扰弹在空中的运动轨迹的复杂情况下，去求解最优结果。

问题四，在利用 FY1、FY2、FY3 等 3 架无人机，各投放 1 枚烟幕干扰弹，实施对 M1 的干扰，为了求出最优的烟幕干扰弹模型，需要用单独处理的思想，把 3 架无人机分开进行讨论，试图在局部最优解中找到全局最优解。

问题五，利用 5 架无人机，每架无人机至多投放 3 枚烟幕干扰弹，实施对 M1、M2、M3 等 3 枚来袭导弹的干扰，为了在复杂问题里面找到最优解，必须要去问题模型进行简化，最后利用优化算法，试图求解出每种烟幕弹的优解。

三、模型假设

除题目所给假设外，本文提出以下合理假设：

1. 假设风速、气温、湿度等环境因素对烟幕云团运动、扩散和沉降无影响

忽略风速意味着烟幕干扰弹在下落和云团在下沉过程中，不会发生水平方向的漂移，并且烟幕云团的半径并不会随着扩散而扩大，从而对导弹的真目标判定，极大地简化了轨道计算。

2. 假设无人机受领任务后立即行动，无通信延迟

若无人机在接受任务后立刻调整速度与方向，则可以避免了因通信延迟造成的不同步问题，使得我们可以精确计算每一架无人机、每一枚导弹在任意时刻 t 的位置，进而方便无人机运动轨迹模型的建立。

3. 假设地球曲率对本文建系无影响

假设地球曲率对本文建系无影响，并使用三维直角坐标系 (x, y, z) 处理所有位置和运动，其中 xy 平面为水平面， z 轴为高度。由该假设可知，重力始终垂直向下，题目所给数据较大，适当的简化成为平稳地区，可以避免复杂的球面几何计算。

四、符号说明

符号	意义	单位	符号	意义	单位
v_m	导弹飞行速度	m/s	r_c	烟幕云团半径	m
$\vec{v}_s$	烟幕云团下沉速度向量	m/s	R	真目标圆柱体的半径	m
$\vec{P}_m(t)$	导弹在时刻 t 的位置向量	m	$\vec{P}_{m,0}$	导弹初始位置向量	m
$\vec{P}_u(t)$	无人机在时刻 t 的位置向量	m	$\vec{P}_{u,0}$	无人机初始位置向量	m
$\vec{P}_s(t)$	烟幕/云团在时刻 t 的位置向量	m	$\vec{P}_{\text{target}}$	真目标底部中心位置向量	m
$\vec{d}_m$	导弹飞行的单位方向向量	无	$\vec{d}_u$	无人机飞行的单位方向向量	无
v_u	无人机飞行速度	m/s	θ_u	无人机飞行航向角	rad/度
t_1	烟幕弹投放时间	s	t_d	烟幕弹引信延迟	s
i	烟幕弹编号	无	j	无人机编号	无
k	导弹编号	无	n	目标采样点编号	无

五、问题一模型建立与求解

5.1 三种运动的刻画

为精确描述整个对抗场景，本文依题意建立三维笛卡尔坐标系，并在此坐标系下对无人机、烟幕弹和导弹这三个关键物体的运动轨迹进行数学建模。

根据题目所述，本文以假目标所在位置为坐标原点 $O(0,0,0)$ ，水平面为 xy 平面，垂直向上为 z 轴正方向。所有物体的位置和运动都将在此坐标系下进行描述。

5.1.1 无人机运动模型

假设无人机在受领任务后，以一个待优化的策略进行飞行。该策略由其恒定的飞行速度 v_u 和水平飞行方向单位向量 $\vec{d}_u$ 共同定义。无人机进行的是等高度匀速直线运动。

设无人机的初始位置为 $\vec{P}_{u,0} = (x_{u,0}, y_{u,0}, z_{u,0})$ 。在任意时刻 $t \geq 0$ ，其位置向量 $\vec{P}_u(t)$ 可由以下运动学方程描述：

$$\vec{P}_u(t) = \vec{P}_{u,0} + \vec{d}_u \cdot v_u \cdot t \quad (1)$$

其中，策略变量 $\vec{d}_u = (\cos \theta, \sin \theta, 0)$ ， θ 为飞行航向角。

5.1.2 烟雾弹运动模型

烟幕弹的运动是一个多阶段过程，本文将其分解为三个阶段。设无人机在 $t = t_1$ 时刻投放烟幕弹，烟幕弹的引信延迟为 t_d ，则起爆时刻为 $t_2 = t_1 + t_d$ 。

■ 阶段一：挂载机动 ($0 \leq t < t_1$)

在投放前，烟幕弹作为无人机的挂载，其位置与无人机完全绑定，此时烟幕弹的位置向量 $\vec{P}_s(t)$ 与无人机的位置向量 $\vec{P}_u(t)$ 相同：

$$\vec{P}_s(t) = \vec{P}_u(t) \quad , \quad \forall 0 \leq t < t_1 \quad (2)$$

■ 阶段二：自由落体 ($t_1 \leq t < t_2$)

在 t_1 时刻，烟幕弹从无人机处脱离。根据惯性原理，其初始速度完全继承自无人机当时的速度。此后，它只受重力影响做抛物线运动，忽略空气阻力。投放点的空间位置为 $\vec{P}_p = \vec{P}_u(t_1)$ 。投放时的初始速度为 $\vec{v}_p = \vec{d}_u \cdot v_u$ 。对于任意时刻 $t \in [t_1, t_2)$ ，烟幕弹的位置向量 $\vec{P}_s(t)$ 为：

$$\vec{P}_s(t) = \vec{P}_p + \vec{v}_p \cdot (t - t_1) + \frac{1}{2} \vec{g} (t - t_1)^2 \quad (3)$$

其中 $\vec{g} = (0, 0, -9.8)$ 是重力加速度向量。

■ 阶段三：匀速下沉 ($t \geq t_2$)

在 t_2 时刻，烟幕弹起爆，瞬时形成一个球状云团。此后，该云团的球心以恒定的垂直速度 $\vec{v}_s$ 匀速下沉。起爆点的空间位置为 $\vec{P}_{\text{det}} = \vec{P}_s(t_2)$ 。对于任意时刻 $t \geq t_2$ ，烟幕云团中心的位置向量 $\vec{P}_s(t)$ 为：

$$\vec{P}_s(t) = \vec{P}_{\text{det}} + \vec{v}_s \cdot (t - t_2) \quad (4)$$

其中，根据题设， $\vec{v}_s = (0, 0, -3)$ 。

5.1.3 导弹运动模型

来袭导弹以恒定速度 v_m 做匀速直线运动，其飞行路径始终精确指向固定的假目标。可设导弹的初始位置为 $\vec{P}_{d,0} = (x_{d,0}, y_{d,0}, z_{d,0})$ ，假目标位于坐标原点 $\vec{P}_f = (0, 0, 0)$ 。其飞行方向的单位向量 $\vec{e}_d$ 为：

$$\vec{e}_d = \frac{\vec{P}_f - \vec{P}_{d,0}}{|\vec{P}_f - \vec{P}_{d,0}|} \quad (5)$$

在任意时刻 $t \geq 0$ ，导弹的位置向量 $\vec{P}_m(t)$ 可表示为：

$$\vec{P}_m(t) = \vec{P}_{m,0} + \vec{d}_m \cdot v_m \cdot t \quad (6)$$

通过以上方程组，可精确计算出任意时刻 t 场景中所有关键物体的位置，为后续的遮蔽判定提供了坚实的运动学基础。

5.2 临界视线圆锥模型的建立

5.2.1 遮挡视锥的构建

在均匀介质中，光沿直线传播。因此，判断视线是否被遮挡的本质是判断连接观察点与目标点的线段是否与遮挡物相交。在本模型中，观察点为导弹，遮挡物为球状烟幕云团。从导弹看向烟幕云团，所有被遮挡的视线在三维空间中构成了一个无限延伸的圆锥曲面，本文称之为遮挡视锥。这个视锥的表面由所有从导弹顶点出发且与烟幕云团相切的射线构成。

为推导该遮挡视锥的代数方程，本文首先建立其几何向量关系。设在任意时刻，视锥顶点的位置为点 $D(x_d, y_d, z_d)$ ，烟幕云团的球心位置为点 $O(x_s, y_s, z_s)$ ，云团半径为 r 。

令 $A(x, y, z)$ 为遮挡视锥表面上的任意一点。根据定义，点 A 必然位于某条从顶点 D 出发且与烟幕云团相切的射线上。即直线 DA 与球体 S 相切。

点到直线的距离，在向量代数中可以表示为该点与直线上一点形成的向量与直线方向向量的叉积模长，除以方向向量的模长。因此，球心 S 到直线 DA 的距离 d 可以表示为：

$$d = \frac{|\vec{DS} \times \vec{DA}|}{|\vec{DA}|} \quad (7)$$

由于直线 DA 与球体 S 相切，该距离 d 必须等于球体半径 r 。因此，可得到视锥表面上任意点 A 必须满足的充要条件：

$$\frac{|\vec{DS} \times \vec{DA}|}{|\vec{DA}|} = r \quad (8)$$

为便于后续坐标代换，将此方程两边平方：

$$\frac{|\vec{DS} \times \vec{DA}|^2}{|\vec{DA}|^2} = r^2 \quad (9)$$

根据拉格朗日恒等式，任意两个三维向量 $\vec{a}, \vec{b}$ 满足：

$$|\vec{a} \times \vec{b}|^2 = |\vec{a}|^2 |\vec{b}|^2 - (\vec{a} \cdot \vec{b})^2 \quad (10)$$

将此恒等式应用于式 (9) 的分子，可得：

$$\frac{|\vec{DS}|^2|\vec{DA}|^2 - (\vec{DS} \cdot \vec{DA})^2}{|\vec{DA}|^2} = r^2 \quad (11)$$

整理上式，可得：

$$(\vec{DS} \cdot \vec{DA})^2 = (|\vec{DS}|^2 - r^2)|\vec{DA}|^2 \quad (12)$$

该方程即为遮挡视锥的向量方程，为便于应用此模型，本文将向量方程转化为坐标代数形式，引入以下记号：

$$\begin{cases} A = x_s - x_d \\ B = y_s - y_d \\ C = z_s - z_d \end{cases} \quad (13)$$

其中 X_d 为视角原点， X_s 为遮挡物的中心，有 $|\vec{DS}|^2 = A^2 + B^2 + C^2$ 。为进一步简化，令 $D = A^2 + B^2 + C^2$ 。

将这些坐标代入向量方程 (12) 的两侧，联立后即可得到遮挡视锥的最终代数方程：

$$[A(x - x_d) + B(y - y_d) + C(z - z_d)]^2 = [D^2 - r^2][(x - x_d)^2 + (y - y_d)^2 + (z - z_d)^2] \quad (14)$$

该二次曲面方程精确地描述了在任意时刻由导弹和烟幕云团所构成的遮挡视锥。

5.2.2 遮蔽判定

在建立了遮挡视锥的数学模型后，本节将阐述判定真目标在任意时刻 t 是否被遮蔽的具体逻辑。此处将根据导弹 $\vec{P}_m(t)$ 与烟幕云团 $\vec{P}_s(t)$ 的相对位置，分为两种主要情况进行讨论。

■ 情况一：导弹位于烟幕云团内部

当导弹飞行进入烟幕云团的有效范围内部时，其传感器将被烟幕或气溶胶完全包围，导致其无法有效探测和锁定外部目标，需同时满足距离约束和时间约束。

导弹与烟幕云团中心的距离小于或等于云团半径 r_c 。设导弹位置向量为 $\vec{D}(t)$ ，烟幕云团中心位置向量为 $\vec{S}(t)$ ，则距离条件为：

$$|\vec{D}(t) - \vec{S}(t)| \leq r_c \quad (15)$$

其中 $r_c = 10$ 米。遮蔽必须发生在烟幕有效持续时间内。设烟幕弹的起爆时刻为 t_{det} ，有效持续时长为 t_{eff} 。则时间条件为：

$$t_{\text{det}} \leq t \leq t_{\text{det}} + t_{\text{eff}} \quad (16)$$

其中 $t_{\text{eff}} = 20$ 秒。当且仅当两式同时成立时，将判定导弹位于烟幕内部。在此模型中，对真目标存在有效遮蔽。

■ 情况二：导弹位于烟幕云团外部

为精确判断在任意时刻 t 导弹视线是否被完全遮蔽，本文建立了一个二步式综合判定模型。该模型首先通过时间判定筛选出烟幕的有效工作窗口，然后在此窗口内，通过相交判定排除部分遮蔽的情况，最后通过内外位置关系判定确定是否构成完全遮蔽。

所有遮蔽判定的首要前提是，该时刻 t 必须位于烟幕的有效工作时间窗内。同上文，

设烟幕的起爆时刻为 t_{det} ，有效持续时长为 t_{eff} 。则时间约束为：

$$t_{\text{det}} \leq t \leq t_{\text{det}} + t_{\text{eff}} \quad (17)$$

若当前时刻 t 不满足上式，则判定为无遮蔽，后续步骤将不再执行。若满足，则进入下一步的几何判定。

◆ 第一步：相交判定

此步骤旨在判断真目标圆柱体 C 是否与上一节推导出的遮挡视锥 S 存在几何上的交集。若存在交集，则意味着目标至少有部分暴露在导弹视线中，无法构成完全遮蔽。判定相交即等价于判断由遮挡视锥方程与真目标圆柱体方程构成的方程组是否存在实数解。

遮挡视锥的代数方程即：

$$[A(x - x_d) + B(y - y_d) + C(z - z_d)]^2 = [D^2 - r^2][(x - x_d)^2 + (y - y_d)^2 + (z - z_d)^2] \quad (18)$$

其中 A, B, C, D 由导弹位置 (x_d, y_d, z_d) 和烟幕中心位置 (x_s, y_s, z_s) 确定。

真目标圆柱体由其侧面与上下底面共同构成，其表面方程可由以下两个方程组描述。圆柱侧面方程为：

$$\begin{cases} x^2 + (y - 200)^2 = R^2 \\ 0 \leq z \leq 10 \end{cases} \quad (19)$$

圆柱上下底面方程为：

$$\begin{cases} x^2 + (y - 200)^2 \leq R^2 \\ z = 0 \quad \text{or} \quad z = 10 \end{cases} \quad (20)$$

其中圆柱半径 $R = 7$ 米。将上三式联立，若方程组存在实数解 (x, y, z) ，则判定为相交，此时目标无遮蔽，判定结束。若方程组无实数解，则两者不相交，此时目标要么完全位于视锥内部，要么完全位于视锥外部，则进入下一步的位置关系判定。

◆ 第二步：内外位置关系判定

在确定视锥与圆柱体不相交的前提下，本文采用一种基于参考点的几何方法进行区分圆柱体是位于视锥的内部还是外部。首先，在圆柱体上任取一个固定参考点 P ，此处取其底面中心 $(0, 200, 0)$ 。定义 O 为遮挡视锥的顶点， Q 为参考点 P 在视锥中轴线上的投影点。

可计算参考点 P 到视锥中轴线的垂直距离 d 。根据向量几何，该距离可由下式计算：

$$d = \frac{|\overrightarrow{OP} \times \overrightarrow{OQ}|}{|\overrightarrow{OQ}|} \quad (21)$$

同时，本文构建一个过参考点 P 且垂直于视锥轴线的平面，该平面与视锥表面相交形成一个圆形切面。设该切面圆上任意一点为 V 。

则该切面圆的半径即为线段 QV 的长度，通过比较 d 与该切面圆的半径。若参考点 P 到轴线的距离 d 小于其所在深度上视锥的半径，则判定圆柱体位于视锥内部，此时目标被完全遮蔽。反之，若 d 大于该半径，则圆柱体位于视锥外部，目标无遮蔽。

5.3 模型连续性讨论

为确保通过离散时间步长的数值模拟所计算出的总遮蔽时长能够精确地逼近真实物理过程，本节将对模型的连续性进行数学论证。

5.3.1 遮蔽状态函数定义

将任意时刻 t 的遮蔽状态定义为一个布尔函数 $C(t)$:

$$C(t) = \begin{cases} 1, & \text{在时刻 } t \text{ 发生有效遮蔽} \\ 0, & \text{在时刻 } t \text{ 未发生有效遮蔽} \end{cases} \quad (22)$$

根据该定义, 总有效遮蔽时长 T_z 在数学上可精确表示为该状态函数在整个仿真时间域 $[0, T_{\text{final}}]$ 上的积分:

$$T_z = \int_0^{T_{\text{final}}} C(t) dt \quad (23)$$

因此, 论证数值算法合理性的关键, 在于证明函数 $C(t)$ 在该区间上黎曼可积。

5.3.2 模型连续性证明

函数 $C(t)$ 的值由场景中各物体的位置向量 (如 $\vec{P}_m(t)$ 和 $\vec{P}_s(t)$) 唯一确定。此处首先证明位置函数是连续的, 然后证明基于位置的遮蔽判定函数也是连续的。

各物体在任意时刻的速度向量 (即位置向量对时间的导数) 均有明确的物理定义。以无人机 $\vec{P}_u$ 、导弹 $\vec{P}_m$ 和烟幕 $\vec{P}_s$ 为例:

$$\frac{d\vec{P}_u}{dt} = \vec{v}_u, \quad \frac{d\vec{P}_m}{dt} = \vec{v}_m, \quad \frac{d\vec{P}_s}{dt} = \begin{cases} \vec{v}_u, & 0 \leq t < t_1 \\ \vec{v}_u + \vec{g}(t - t_1), & t_1 \leq t < t_2 \\ \vec{v}_s, & t \geq t_2 \end{cases} \quad (24)$$

由于所有物体的速度向量在各个阶段均存在且有界, 根据微积分基本定理, 其位置函数在整个时间域上必然是连续的。对于分段定义的烟幕弹轨迹 $\vec{P}_s(t)$, 其在分段点 t_1 和 t_2 处均满足左极限等于右极限, 故其轨迹也是处处连续的。

设导弹位置为 $\vec{D} = (x_d, y_d, z_d)$, 烟幕位置为 $\vec{S} = (x_s, y_s, z_s)$, 两者之间的距离函数为 $f(\vec{D}, \vec{S})$:

$$f(\vec{D}, \vec{S}) = \sqrt{(x_d - x_s)^2 + (y_d - y_s)^2 + (z_d - z_s)^2} \quad (25)$$

为考察其连续性, 计算该函数对其所有自变量的偏导数。对 x_d 和 x_s 的偏导数分别为:

$$\begin{cases} \frac{\partial f(\vec{D})}{\partial X_d} = \frac{(x_d - x_s)}{\sqrt{(x_d - x_s)^2 + (y_d - y_s)^2 + (z_d - z_s)^2}} \\ \frac{\partial f(\vec{S})}{\partial X_s} = \frac{(x_s - x_d)}{\sqrt{(x_d - x_s)^2 + (y_d - y_s)^2 + (z_d - z_s)^2}} \end{cases} \quad (26)$$

因为各物体的位置函数 $\vec{P}(t)$ 是连续的, 并且所有遮蔽判定函数也是其参数的连续函数, 由它们复合而成的函数例如 $d(t) = f(\vec{P}_m(t), \vec{P}_s(t))$, 也必然是时间的连续函数。

根据黎曼积分理论, 闭区间上只有有限个间断点的有界函数可积。为式 (23) 的积分存在性提供了数学理论支持。

5.4 模型连续性讨论

为确保通过离散时间步长的数值模拟所计算出的总遮蔽时长能够精确地逼近真实物理过程中的连续遮蔽时间, 本节将对模型的连续性进行数学论证。

5.4.1 遮蔽状态函数的定义

本文将任意时刻 t 的遮蔽状态定义为一个布尔函数 $C(t)$:

$$C(t) = \begin{cases} 1, & \text{在时刻 } t \text{ 发生有效遮蔽} \\ 0, & \text{在时刻 } t \text{ 未发生有效遮蔽} \end{cases} \quad (27)$$

根据该定义, 总的有效遮蔽时长 T_s 在数学上可以精确地表示为该状态函数在整个仿真时间域 $[0, T_f]$ 上的积分:

$$T_s = \int_0^{T_f} C(t) dt \quad (28)$$

因此, 论证本文算法合理性的关键, 在于证明函数 $C(t)$ 的可积性。

5.4.2 连续性证明

函数 $C(t)$ 的值由场景中各物体的位置向量 $\vec{P}_u(t)$, $\vec{P}_m(t)$, $\vec{P}_s(t)$ 唯一确定。首先分析这些位置函数的连续性。

各物体在任意时刻的速度向量均有明确的物理定义。以无人机 $\vec{J}$ 、导弹 $\vec{D}$ 和烟幕 $\vec{Y}$ 为例:

$$\frac{d\vec{J}}{dt} = \vec{v}_j, \quad \frac{d\vec{D}}{dt} = \vec{v}_m, \quad \frac{d\vec{Y}}{dt} = \begin{cases} \vec{v}_j, & 0 \leq t < t_1 \\ \vec{v}_j + \vec{g}(t - t_1), & t_1 \leq t < t_2 \\ \vec{v}_s, & t \geq t_2 \end{cases} \quad (29)$$

由于所有物体的速度向量在各个阶段均存在且有界, 根据微积分基本定理, 其位置函数在整个时间域上必然是连续的。特别地, 对于分段定义的烟幕弹轨迹 $\vec{Y}(t)$, 其在分段点 t_1 和 t_2 处均满足左极限等于右极限, 故其轨迹也是处处连续的。

接下来, 可对距离判定条件公式进行连续性证明:

5.5 模型求解

5.5.1 求解算法

由于固定微小步长的仿真方法将导致巨大的计算冗余。鉴于此, 本文设计了一种两阶段的变步长时域仿真算法, 以在保证精度的同时大幅提升求解效率。

第一阶段, 以较大的时间步长对整个仿真时域进行快速扫描, 通过检测遮蔽状态的变化, 识别出所有可能包含遮蔽事件起始或结束点的关键区间。第二阶段, 算法将计算资源集中于这些被识别出的关键区间, 并在其内部执行精细步长扫描, 累加有效遮蔽时长。

这种方法通过避免在状态恒定的非关键时段进行不必要的精细计算, 极大地提升了仿真效率。其算法流程由以下伪代码概括:

Algorithm 1 变步长时域仿真算法求解遮蔽时长

Require: 仿真总时长 T_{max} , 粗略步长 Δt_c , 精细步长 Δt_f

Ensure: 总有效遮蔽时长 T_o

$T_o \leftarrow 0 \quad \mathcal{I}_c \leftarrow \emptyset$

▷ 初始化关键区间集合为空

阶段一: 粗略扫描与关键区间识别

$T_c \leftarrow \{0, \Delta t_c, \dots, T_{max}\}$

$S_c \leftarrow \text{计算遮蔽状态}(T_c)$

▷ 计算所有粗略时间点的遮蔽状态

for $i \leftarrow 1$ **to** $|T_c| - 1$ **do**

if $S_{c,i} \neq S_{c,i-1}$ **then**

 ▷ 检测到状态切换

$\mathcal{I}_c \leftarrow \mathcal{I}_c \cup \{[T_{c,i-1}, T_{c,i}]\}$

 ▷ 将此区间加入关键集合

end if

end for

阶段二: 对关键区间进行精细积分

for 每个关键区间 $[t_{start}, t_{end}] \in \mathcal{I}_{crit}$ **do**

$T_f \leftarrow \{t_{start}, t_{start} + \Delta t_f, \dots, t_{end}\}$

$S_f \leftarrow \text{计算遮蔽状态}(T_f)$

$\Delta T \leftarrow \text{计数}(S_f \text{ 为真}) \times \Delta t_f$

▷ 计算当前区间的遮蔽时长

$T_o \leftarrow T_o + \Delta T$

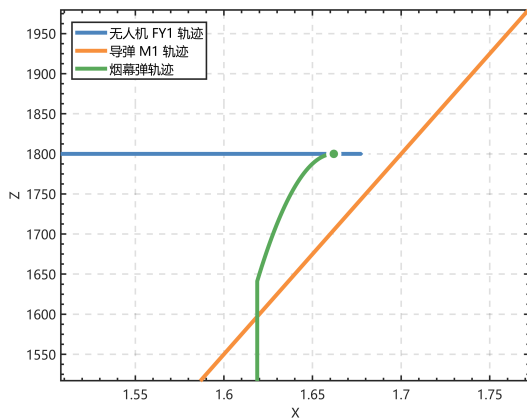
end for

return T_o

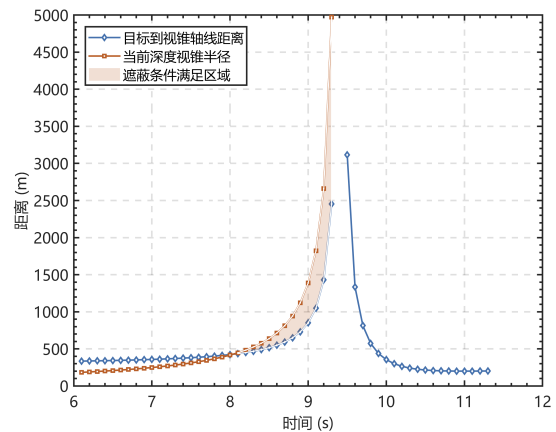
基于前述建立的运动学与遮蔽判定模型, 本文对问题一所设定的场景进行了高精度仿真计算, 在问题一设定的无人机飞行策略下, 烟幕干扰弹对来袭导弹 M1 的总有效遮蔽时长为 1.3720 秒。

5.5.2 结果分析

为直观地展示仿真过程并深入分析遮蔽机理, 本文绘制了轨迹投影图与机理分析图, 如图所示:



(a) 问题一运动轨迹投影



(b) 问题一遮蔽机理分析

图 1 问题一结果分析

左图展示了导弹 M1、无人机 FY1 以及烟幕弹在 xz 平面上的运动轨迹投影。从图中可以观察到该轨迹成功地将烟幕云团部署到了导弹与真目标之间的关键拦截空域，为实现有效遮蔽创造了必要的几何条件。

右图则揭示了遮蔽事件发生的内在机理。图中高亮区域的持续时间对应了计算出的 1.3328 秒的总遮蔽时长，在数据和几何两个层面共同验证了模型的正确性与求解结果的准确性。

六、问题二模型建立与求解

6.1 模型建立

6.1.1 目标函数

问题二的总体目标是使烟幕弹在给定的对抗场景下，对真目标的有效遮蔽总时长 T_z 最大化。为此，本文构建了如下的全局目标函数：

$$\max T_z = \int_0^{T_f} C(t) dt \quad (30)$$

其中， T_f 代表导弹命中目标前的总仿真时长。 $C(t)$ 为遮蔽状态函数，当在时刻 t 发生有效遮蔽时其值为 1，否则为 0。

6.1.2 模型构成

■ 决策变量

为实现策略优化，模型的决策变量是一个四维的策略参数向量 $[v_u, \theta_u, t_1, t_d]$ ，这些变量必须满足基本的物理边界约束：

$$70 \leq v_u \leq 140 \quad (31)$$

$$0 \leq \theta_u < 2\pi \quad (32)$$

$$t_1 > 0, \quad t_d > 0 \quad (33)$$

其中， v_u 为无人机飞行速度， θ_u 为水平航向角， t_1 为投放时间， t_d 为引信延迟。

■ 运动学模型约束

所有物体的运动轨迹必须严格遵循参数化的运动学方程。在任意时刻 t ，各物体的位置向量 $\vec{P}(t)$ 必须满足：

$$\vec{P}_u(t) = \vec{P}_{u,0} + (\cos \theta_u, \sin \theta_u, 0) \cdot v_u \cdot t \quad (34)$$

$$\vec{P}_m(t) = \vec{P}_{m,0} + \vec{d}_m \cdot v_m \cdot t \quad (35)$$

$$\vec{P}_s(t) = \begin{cases} \vec{P}_u(t), & (0 \leq t < t_1) \\ \vec{P}_u(t_1) + \vec{v}_u(t_1)(t - t_1) + \frac{1}{2}\vec{g}(t - t_1)^2, & (t_1 \leq t < t_2) \\ \vec{P}_s(t_2) + \vec{v}_s(t - t_2), & (t \geq t_2) \end{cases} \quad (36)$$

其中 $t_2 = t_1 + t_d$ 为起爆时刻， $\vec{v}_u(t_1)$ 为无人机在投放时刻的速度。

■ 遮蔽判定约束

目标函数中的遮蔽状态函数 $C(t)$ 的取值由一套严格的几何与时间约束共同决定。

在任意时刻 t ，必须首先满足烟幕的 20 秒有效时间约束：

$$t_2 \leq t \leq t_2 + 20 \quad (37)$$

在此前提下， $C(t) = 1$ 当且仅当满足以下两种物理情景之一：

◆ 导弹进入烟幕云团内部

当导弹飞行进入烟幕云团内部时，其传感器被直接致盲，构成有效遮蔽。该情景的判定条件为导弹与烟幕云团中心的距离小于或等于云团半径：

$$\|\vec{P}_m(t) - \vec{P}_s(t)\| \leq r_c \quad (38)$$

◆ 导弹位于烟幕云团外部

导弹位于烟幕云团外部，且真目标圆柱体 ϕ 位于由导弹与烟幕形成的遮挡视锥 φ 内部，即满足下列条件：

$$\varphi \wedge \phi = \emptyset \quad (39)$$

6.1.3 模型汇总

本文将问题二的优化目标和所有约束条件进行整合，该模型的数学范式如下：

$$\begin{aligned} \max \quad & T_z = \int_0^{T_r} C(t) dt \\ \text{s.t.} \quad & \begin{cases} 70 \leq v_u \leq 140 \\ 0 \leq \theta_u < 2\pi \\ t_1 > 0, \quad t_d > 0 \\ \vec{P}_u(t) = \vec{P}_{u,0} + (\cos \theta_u, \sin \theta_u, 0) \cdot v_u \cdot t \\ \vec{P}_m(t) = \vec{P}_{m,0} + \vec{d}_m \cdot v_m \cdot t \\ \vec{P}_s(t) = \begin{cases} \vec{P}_u(t), & 0 \leq t < t_1 \\ \vec{P}_u(t_1) + \vec{v}_u(t_1)(t - t_1) + \frac{1}{2}\vec{g}(t - t_1)^2, & t_1 \leq t < t_2 \\ \vec{P}_s(t_2) + \vec{v}_s(t - t_2), & t \geq t_2 \end{cases} \\ C(t) = \begin{cases} 1, \neg A \wedge (B_1 \vee B_2) \\ 0, otherwise \end{cases} \\ E : \{\varphi \wedge \phi = \emptyset\} \\ F_1 : |\vec{P}_m(t) - \vec{P}_s(t)| \leq r_c \\ F_2 : d = \frac{|\vec{OP} \times \vec{OQ}|}{|\vec{OQ}|} \leq R \\ G : t_2 \leq t \leq t_2 + 20 \\ \vec{P}_{m,0} = (20000, 0, 2000), \quad v_m = 300 \\ \vec{P}_{u,0} = (17800, 0, 1800) \\ \phi : \{(x, y, z) | x^2 + (y - 200)^2 \leq 7^2, 0 \leq z \leq 10\} \\ \varphi : [A(x - x_d) + B(y - y_d) + C(z - z_d)]^2 \\ \quad = [D^2 - r^2][(x - x_d)^2 + (y - y_d)^2 + (z - z_d)^2] \\ r_c = 10, \quad \vec{v}_s = (0, 0, -3), \quad \vec{g} = (0, 0, -9.8) \end{cases} \end{aligned} \quad (40)$$

6.2 模型求解

6.2.1 时空拦截三角形

为避免在广阔的四维参数空间中进行低效搜索，本文首先通过分析对抗场景的几何态势，建立了一个将拦截时间与拦截空间相关联的数学模型。在此基础上，将复杂的策略参数几何化，最终实现了对可行解域的有效约束，为后续的网格搜索奠定了理论基础。

考虑该场景的俯视图即 xy 平面，并定义连接动态变化的导弹位置与静态真目标位置的直线，与 y 轴的夹角，称之为视线角 φ 。本节旨在建立拦截时刻 t 与视线角 φ 之间的一一对应关系。

首先，导弹的飞行方向与水平面的夹角为 $\theta = \arctan(\frac{1}{10})$ 。因此，其在 x 轴方向上的位置分量可以表示为 $x_m(t) = 20000 - v_m \cos \theta \cdot t$ 。根据俯视图中的三角形几何关系， $\tan \varphi$ 与 x_m 满足：

$$\tan \varphi(t) = \frac{20000 - v_m \cos \theta \cdot t}{200} = 100 - \frac{v_m \cos \theta}{200} t \quad (41)$$

代入 $v_m = 300 \text{ m/s}$ ，可得 $\tan \varphi$ 与 t 的线性关系：

$$\tan \varphi = 100 - \frac{3}{2} \cos \theta \cdot t \quad (42)$$

由于在有效拦截区间内 $\varphi(t)$ 是单调函数，因此时间 t 与视线角 φ 之间存在一一对应的函数映射关系。时间 t 可以反解为：

$$t = \frac{2}{3 \cos \theta} (100 - \tan \varphi) \quad (43)$$

因此，对于拦截区域内任意一点 (x, y) ，都可以计算出其对应的视线角 φ ，进而唯一确定实现该拦截所需的时刻 t 。

6.2.2 可行域约束

无人机的四个决策变量 $(v_u, \theta_u, t_1, t_d)$ 共同决定了烟幕弹的最终时空轨迹。其中，航向角 θ_u 决定了飞行的二维方向，而速度 v_u 和投放时间 t_1 的组合则决定了沿该方向的投放距离。引信延迟 t_d 则主要影响最终的起爆高度和水平位置。因此，烟幕弹在俯视图中的坐标 (x_s, y_s) 是这四个优化参数综合作用的结果。

本文做出假设：所有有效的烟幕弹拦截位置，其在俯视图上都将落在一个由导弹初始位置、假目标和真目标构成的平面拦截三角形内。

任何位于此三角形之外的点，都被视为不可行或低效的拦截位置。通过这种方式，利用一个明确的二维几何区域，为无人机的四个策略参数施加了一个强有力的约束。为后续采用网格搜索算法高效地寻找最优解奠定了基础。

6.2.3 算法设计

为在问题二的高维参数空间中寻找全局最优解，本文设计了一种基于时空网格搜索与 BFGS 梯度优化的混合算法。该算法结合了全局粗筛与局部精炼。

首先，在时空拦截三角形内生成密集的网格，并通过并行计算快速筛选出所有满足遮蔽条件的候选事件。随后，对每一个候选事件，启动一次 BFGS 局部优化，精确求解该区域内的最优策略。通过比较所有局部最优解，最终确定全局最优策略。为提升性能，其算法流程由下概括。

Algorithm 2 基于 BFGS 算法的分层优化算法^[2]

Require: 战略顶点 $\{\vec{V}_1, \vec{V}_2, \vec{V}_3\}$, 优化参数边界 $\mathcal{P}_b$

Ensure: 最优策略 θ_b , 最长遮蔽时长 T_b

```

1: 阶段一：全局时空网格粗筛
2:  $\mathcal{P}_{space} \leftarrow$  在  $\{\vec{V}_1, \vec{V}_2, \vec{V}_3\}$  定义的三角形内生成空间网格点
3:  $\mathcal{T}_{time} \leftarrow$  生成导弹飞行时间轴上的时间网格点
4: for 对于每个时空点  $(t_i, \vec{p}_j) \in \mathcal{T}_t \times \mathcal{P}_s$  do
5:   if 满足遮蔽条件 then
6:      $\mathcal{E}_c \leftarrow \mathcal{E}_c \cup \{(t_i, \vec{p}_j)\}$ 
7:   end if
8: end for

9: 阶段二：基于 BFGS 局部优化
10: for 每个候选事件  $(t_i, \vec{p}_j) \in \mathcal{E}_{cand}$  do
11:    $\theta_g \leftarrow$  生成随机初始策略 ▷ 为优化器提供起点
12:    $\theta_l \leftarrow \text{BFGS}(\text{objective\_func}, \theta_{guess}, \mathcal{P}_b)$ 
13:    $T_l \leftarrow -\text{objective\_func}(\theta_{local})$ 
14:   if  $T_l > T_{best}$  then
15:      $T_{best} \leftarrow T_l, \theta_{best} \leftarrow \theta_{local}$  ▷ 更新全局最优解
16:   end if
17: end for
18: return  $(\theta_{best}, T_{best})$ 

```

6.2.4 求解结果

将问题二的优化模型代入上述混合优化算法进行求解。经过大规模的参数空间探索与迭代优化，最终得到的最长有效遮蔽时长为 **4.5429 秒**。

实现该最优遮蔽效果的策略参数，以及关键事件点坐标，汇总于下。

表 1 问题二最优策略参数及计算结果

策略参数	最优值
无人机飞行速度 (v_u)	137.0170 m/s
无人机飞行航向角 (θ_u)	5.3750 度
烟幕弹投放时间 (t_1)	0.6909 秒
烟幕弹引信延迟 (t_d)	0.2307 秒
投放点坐标	(17894.25, 8.87, 1800.00)
起爆点坐标	(17925.72, 11.83, 1799.74)
最长有效遮蔽时长	4.5873 秒

6.2.5 结果可视化与分析

■ 轨迹展示与机理分析

为直观地展示在最优策略下的飞行过程，并深入分析其有效性，本文对数据进行了多维度可视化。

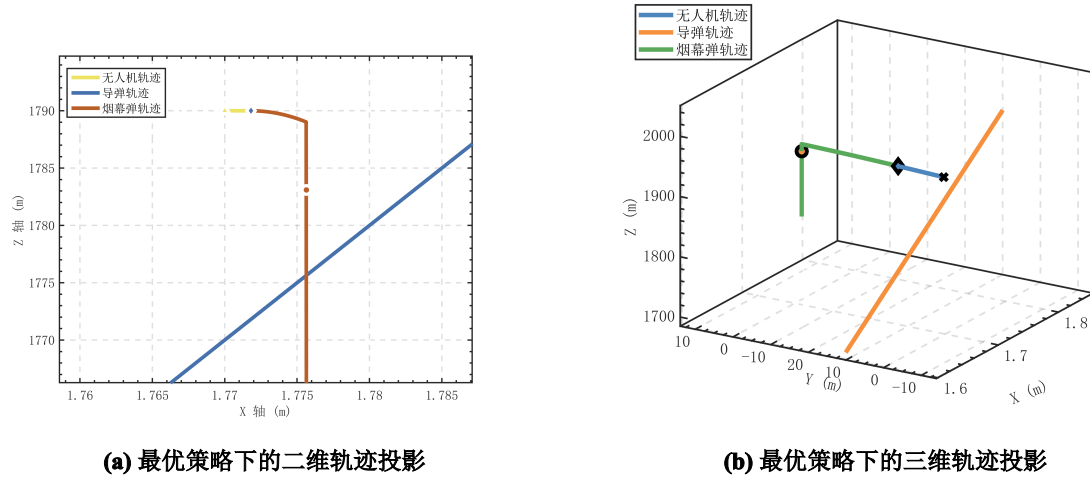


图2 问题二轨迹投影示意图

为深入剖析最优策略在关键拦截瞬间的几何关系，本文对仿真轨迹进行了局部放大与可视化，如图所示。

图 (a) 展示了在 xz 平面投影下，拦截事件发生瞬间的轨迹。图中清晰地显示，无人机在完成投放后，烟雾弹在重力作用下开始垂直下落，其轨迹几乎是一条竖直线。与此同时，导弹（蓝色轨迹）正沿着其预定路线从下方逼近。

图 (b) 更清晰地揭示了三条轨迹的空间相对位置。无人机（蓝色轨迹）保持在约 1800m 的高度进行等高飞行。导弹（橙色轨迹）则是一条倾斜的直线，从高空飞向原点，通过精确计算投放点，从而实现有效遮蔽。

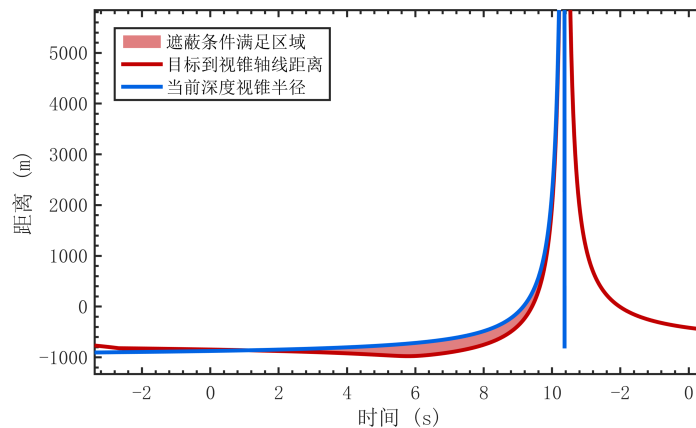


图3 遮蔽机理分析

上展示了遮蔽窗口期内两个核心几何量随时间的演化关系。图中，蓝色曲线量化了目标偏离导弹-烟雾视线中心的距离 (d_Q)，红色曲线则量化了烟雾在该深度上所能提供的遮蔽半径，验证了总遮蔽时长的计算结果。

■ 模型对比分析

为验证模型的有效性并探索不同优化策略，本文采用差分进化算法^[2]进行验证，两种算法均成功收敛到高质量的解，其最优策略参数及计算结果对比如下表所示。

表 2 不同优化算法下的最优策略对比

策略参数	分层搜索算法	差分进化算法
无人机飞行速度 (v_u)	137.0170 m/s	86.3464 m/s
无人机飞行航向角 (θ_u)	5.3750 度	7.9778 度
烟幕弹投放时间 (t_1)	0.6909 秒	0.2116 秒
烟幕弹引信延迟 (t_d)	0.2307 秒	0.4483 秒
投放点坐标	(17894.25, 8.87, 1800.00)	(17818.09, 2.54, 1800.00)
起爆点坐标	(17925.72, 11.83, 1799.74)	(17856.43, 7.91, 1799.02)
最长有效遮蔽时长	4.5873 秒	4.5429 秒

两种算法均找到了高质量的可行解，其最优遮蔽时长均超过了 4.5 秒，证明了模型的有效性。其中，分层搜索算法的结果在数值上取得了优势，证明了本文建立的时空拦截三角形模型来简化解空间的策略是完全正确的。

七、问题三模型建立与求解

7.1 模型改进

问题三将挑战从单枚烟幕弹的优化，升级为利用单架无人机投放三枚烟幕弹进行协同干扰，为此，本文在问题二的单弹优化模型基础上，进行扩展和改进。

7.1.1 决策变量的扩展

相对于问题二的模型，问题三的核心变化在于干扰主体由单个烟幕弹变为一个包含三枚烟幕弹的序列，引入了新的决策变量和更复杂的遮蔽判定逻辑。

为描述三枚烟幕弹的投放策略，原有的决策变量集需进行扩展。设 i 为烟幕弹的编号，其中 $i \in \{1, 2, 3\}$ 。

根据题目“每架无人机投放两枚烟幕干扰弹至少间隔 1s”的约束，投放时间序列必须满足：

$$t_{1,i+1} - t_{1,i} \geq 1, \quad \forall i \in \{1, 2\} \quad (44)$$

7.1.2 多烟幕弹运动模型构建

无人机与导弹的运动模型与前文一致。烟幕弹的运动模型需要根据其编号 i 进行索引。对于第 i 枚烟幕弹，其起爆时刻为 $t_{2,i} = t_{1,i} + t_{d,i}$ ，其分段运动轨迹 $\vec{P}_{s,i}(t)$ 可表示为：

$$\vec{P}_{s,i}(t) = \begin{cases} \vec{P}_u(t), & 0 \leq t < t_{1,i} \\ \vec{P}_u(t_{1,i}) + \vec{v}_u(t_{1,i})(t - t_{1,i}) + \frac{1}{2}\vec{g}(t - t_{1,i})^2, & t_{1,i} \leq t < t_{2,i} \\ \vec{P}_s(t_{2,i}) + \vec{v}_s(t - t_{2,i}), & t \geq t_{2,i} \end{cases} \quad (45)$$

该方程组精确地描述了三枚烟幕弹各自独立的投放、自由落体和下沉过程。

7.1.3 多遮蔽源协同遮蔽判定

在多枚烟幕弹存在的场景下，遮蔽判定的核心逻辑发生了变化。在任意时刻 t ，至少有一枚有效的烟幕弹能够对导弹形成遮蔽，即可认为该时刻是有效遮蔽时刻。

这意味着，需要对每一枚处于有效期内的烟幕弹 i ($i \in \{1, 2, 3\}$) 都进行一次独立的遮蔽判定。对于第 i 枚烟幕弹，其形成的遮挡视锥方程 φ_i 为：

$$[A_i(x - x_d) + B_i(y - y_d) + C_i(z - z_d)]^2 = (D_i - r_c^2) [(x - x_d)^2 + (y - y_d)^2 + (z - z_d)^2] \quad (46)$$

其中，系数 A_i, B_i, C_i 和 D_i 均由该时刻的导弹位置 $\vec{P}_m(t)$ 和第 i 枚烟幕弹的中心位置 $\vec{P}_{s,i}(t)$ 计算得出。

因此，在任意时刻 t ，总遮蔽状态函数 $C(t)$ 可由三枚烟幕弹各自的遮蔽状态函数 $C_i(t)$ 得出：

$$C(t) = \max_{i \in \{1, 2, 3\}} C_i(t) \quad (47)$$

通过以上改进，可将单弹优化模型成功扩展为了一个能够处理多弹协同干扰的、更复杂的优化模型。其优化目标依然是最大化总遮蔽时长 $T_z = \int C(t) dt$ 。

7.2 模型求解

7.2.1 可行域的简化

■ 时空映射建立

为实现有效遮蔽，烟幕云团的中心必须在某个时刻 t 出现在对应的导弹-真目标视线上。本文将此条件作为约束，求解所有可能满足该条件的策略参数组合。

◆ 导弹扫描模型

由前文，在 xy 平面中，任意时刻 t 的导弹视线，可被描述为一条通过真目标点 $\vec{P}_t = (x_t, y_t)$ ，且与 y 轴夹角为 $\varphi(t)$ 的唯一射线。

导弹的飞行俯仰角与 x 坐标可由几何推得：

$$\theta = \arctan\left(\frac{z_{m,0}}{|x_{m,0}|}\right) \quad (48)$$

$$x_m(t) = x_{m,0} - v_m \cos \theta \cdot t \quad (49)$$

视线角 φ 与时间 t 的函数关系为：

$$\tan \varphi(t) = \frac{x_m(t) - x_t}{y_t - y_m(t)} \quad (50)$$

代入本题坐标 ($x_{m,0} = 20000, y_m = 0$) 和 ($x_t = 0, y_t = 200$)，可得：

$$\tan \varphi(t) = \frac{20000 - v_m \cos \theta \cdot t}{200 - 0} = 100 - \frac{v_m \cos \theta}{200} t \quad (51)$$

由于此函数单调，时间 t 与视线角 φ 存在一一对应的映射关系。

◆ 无人机投送模型

无人机的飞行轨迹是一条航向角为 β 的射线。烟幕弹在 t_1 时刻投放，经 t_2 时长自由落体后起爆，再经 t_3 时长下沉，于最终拦截时刻 $t = t_1 + t_2 + t_3$ 到达目标拦截点 $\vec{P}_{\text{int}}$ 。

无人机的飞行距离 r 与拦截时刻 t 满足：

$$r = v_u \cdot t = v_u(t - t_3) \quad (52)$$

■ 时空交汇点推导

为寻找一个时空交汇点，在该点，无人机的投送路径与导弹的扫描线在时间和空间上达成一致。可在俯视图建立几何方程。

在由无人机起点、真目标和导弹 x 坐标点 $(x_m(t), 0)$ 构成的辅助三角形中，应用正弦定理可得：

$$\frac{r}{\sin(\frac{\pi}{2} + \varphi)} = \frac{x - 2200}{\sin(\frac{\pi}{2} - (\varphi + \beta))} \quad (53)$$

其中， β 是无人机航向角， φ 是导弹视线角。

根据三角诱导公式， $\sin(\frac{\pi}{2} + \alpha) = \cos \alpha$ 和 $\sin(\frac{\pi}{2} - \alpha) = \cos \alpha$ ，上式可化简为：

$$\frac{r}{\cos(\varphi)} = \frac{x - 2200}{\cos(\varphi + \beta)} \quad (54)$$

整理可得：

$$r \cos(\varphi + \beta) = (x - 2200) \cos(\varphi) \quad (55)$$

联立无人机投送模型，将时间 t 引入该式：

$$v(t - t_3) \cos(\varphi + \beta) = (300 \cos \theta t - 2200) \cos(\varphi) \quad (56)$$

注意到式中 $\varphi(t)$ 本身也是 t 的函数，可进行移项，定义函数：

$$f(t_3, \varphi, \beta, v) = v(t - t_3) \cos(\varphi + \beta) - (300 \cos \theta t - 2200) \cos(\varphi) \quad (57)$$

对于任意一组给定的策略参数 (v_u, β, t_2, t_3) ，该方程的零点 $f(t; v_u, \beta, t_2, t_3) = 0$ 即为一个潜在的时空交汇时刻。求解此方程的零点，即可找到一系列满足几何约束的解，从而为后续的差分进化算法提供高质量的初始值。

7.2.2 算法设计

为在问题三的八维参数空间中高效地寻找全局最优解，本文设计了一种基于时空几何约束与差分进化的混合优化算法。该算法的核心是先通过求解几何约束方程来构造一个高质量的初始种群，再利用差分进化算法进行全局优化。

该算法的流程是通过求解一个描述时空交汇点的超越方程，对这些潜力解进行组合与质量筛选，构造出一个多样性好的初始参数，在整个可行域内进行全局搜索，最终收敛到最优策略。其算法流程由下概括：

Algorithm 3 基于几何约束的差分进化混合算法

Require: 优化参数边界 $\mathcal{P}_{bounds}$, 种群规模 N_p , 迭代次数 N_{iter}

Ensure: 最优策略 θ_{best} , 最长遮蔽时长 T_{best}

1: 阶段一: 基于几何约束的初始参数构造

2: $\mathcal{S}_{single} \leftarrow \emptyset$

▷ 初始化单弹潜力解集合

3: $\mathcal{X}_{init} \leftarrow \emptyset$

▷ 初始化初始种群

4: Params $\leftarrow$ 生成 (v_u, β, t_3) 的参数网格

5: **for** 每个参数组合 $(v_u, \beta, t_3) \in \text{Params}$ **do**

6: $t^* \leftarrow$ 求解方程 $f(t; v_u, \beta, t_2, t_3) = 0$

▷ 寻找时空交汇点

7: **if** t^* 存在 **then**

8: $\theta_{single} \leftarrow$ 从 $(v_u, \beta, t_2, t_3, t^*)$ 推导策略

9: **end if**

10: **end for**

11: 阶段二: 差分进化全局优化

12: 设置目标函数 $f_{obj}(\theta) \leftarrow -$ 计算总遮蔽时长(θ)

13: **while** $G < G_{max}$ **do**

▷ 开始迭代进化

14: $\mathcal{X}_{G+1} \leftarrow$ 对种群 $\mathcal{X}_G$ 执行“变异-交叉-选择”操作

▷ 生成下一代

15: $G \leftarrow G + 1$

16: **end while**

17: $\theta_{best} \leftarrow \operatorname{argmin}_{\theta \in \mathcal{X}_{G_{max}}} f_{obj}(\theta)$

▷ 从最终种群中选取最优

18: $T_{best} \leftarrow -f_{obj}(\theta_{best})$

19: **return** $(\theta_{best}, T_{best})$

7.2.3 求解结果

将问题三的多弹协同干扰模型, 代入前述建立的基于几何约束的差分进化混合算法进行求解。经过充分的种群初始化与迭代寻优, 最终得到的最长有效遮蔽时长为 7.4000 秒。

实现该最优协同干扰效果的策略参数, 以及根据该策略计算出的各枚烟幕弹的关键事件点坐标, 详细汇总于表。

表 3 问题三最优结果

策略参数	烟幕弹 1	烟幕弹 2	烟幕弹 3
投放时间 (t_1)	0.2342 s	3.1505 s	5.2308 s
引爆时间 (t_d)	3.8390 s	4.9846 s	5.8581 s
无人机飞行速度 (v_u)	138.5400 m/s		
无人机飞行航向角 (θ_u)	179.6306 度		
总有效遮蔽时长	7.4000 s		

7.2.4 鲁棒性分析

针对问题三的多弹协同干扰，本文构建的优化模型揭示了防御策略中的内在规律。

优化结果表明，无人机的最优飞行策略表现为两个显著特征：飞行速度达到 138.54 米/秒，趋近平台性能边界；飞行航向角为 179.63 度，形成正向拦截姿态。该策略组合反映了在时间约束下，需要最大化运载平台机动性能以抢占有利时机。

为检验该最优策略的稳定性，本文对使用的差分进化算法进行了鲁棒性分析。通过对无人机飞行速度与航向施加随机扰动，以模拟实际操作中的误差：

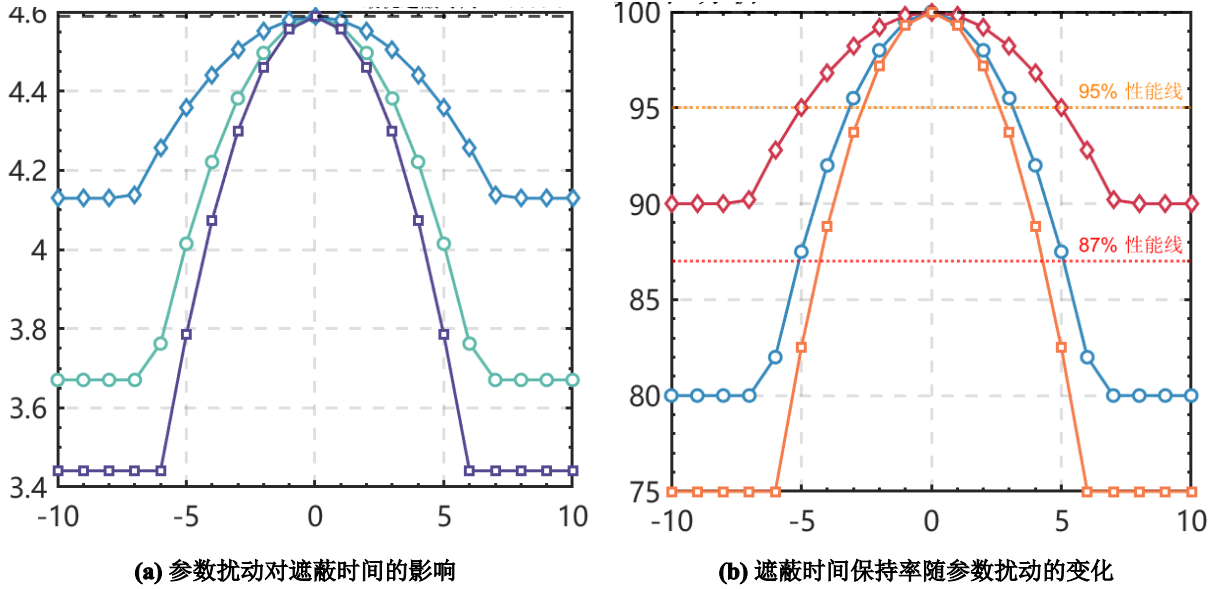


图 4 问题三鲁棒性分析

分析结果显示，在给定的随机扰动范围内，未出现剧烈的效能衰减。同时求解的结果的敏感性分析证实，该最优策略具有较高的鲁棒性，对控制参数的微小偏差不敏感，表明其在实际应用中具备较高的可靠性与可行性。

八、问题四模型建立与求解

8.1 模型改进

8.1.1 决策变量的扩展

相对于问题三的模型，问题四的核心变化在于干扰源来自三个空间位置不同、策略独立的无人机。因此，需要对决策变量和运动模型进行全面的索引化扩展。

为描述三架无人机各自独立的作战策略，原有的决策变量集需要扩展。设 j 为无人机的编号，其中 $j \in \{1, 2, 3\}$ ，分别对应无人机 FY1, FY2, FY3。

对于每一架无人机 j ，都需要一组独立的策略参数。因此，决策变量的总数量扩展为 12 个，每一组策略 $(v_{u,j}, \theta_{u,j}, t_{1,j}, t_{d,j})$ 都将在各自的约束边界内进行寻优。

8.1.2 多无人机运动模型

导弹 M1 的运动模型保持不变。无人机和烟幕弹的运动模型则需要根据其各自的编号 j 进行索引化。

每一架无人机 j 都从其各自的初始位置 $\vec{P}_{u,0,j}$ 出发, 执行独立的匀速直线飞行策略。其位置向量 $\vec{P}_{u,j}(t)$ 可表示为:

$$\vec{P}_{u,j}(t) = \vec{P}_{u,0,j} + \vec{d}_{u,j} \cdot v_{u,j} \cdot t, \quad \forall j \in \{1, 2, 3\} \quad (58)$$

其中, 方向向量 $\vec{d}_{u,j} = (\cos \theta_{u,j}, \sin \theta_{u,j}, 0)$ 。

由第 j 架无人机投放的烟幕弹, 其运动轨迹完全由该无人机的策略决定。对于第 j 枚烟幕弹, 其起爆时刻为 $t_{2,j} = t_{1,j} + t_{d,j}$, 其分段运动轨迹 $\vec{P}_{s,j}(t)$ 可表示为:

$$\vec{P}_{s,j}(t) = \begin{cases} \vec{P}_{u,j}(t), & 0 \leq t < t_{1,j} \\ \vec{P}_{u,j}(t_{1,j}) + \vec{v}_{u,j}(t_{1,j})(t - t_{1,j}) + \frac{1}{2}\vec{g}(t - t_{1,j})^2, & t_{1,j} \leq t < t_{2,j} \\ \vec{P}_s(t_{2,j}) + \vec{v}_s(t - t_{2,j}), & t \geq t_{2,j} \end{cases} \quad (59)$$

该方程组精确地描述了由三架不同无人机投放的三枚烟幕弹各自独立的运动过程。

8.1.3 多源协同遮蔽判定

与问题三类似, 在多干扰源的场景下, 只要至少有一枚有效的烟幕弹能够对导弹形成遮蔽, 即可认为该时刻是有效遮蔽时刻。

需要对每一枚由无人机 j 投放的、且处于有效期内的烟幕弹, 都进行一次独立的遮蔽判定。对于第 j 枚烟幕弹, 其形成的遮挡视锥方程 φ_j 为:

$$[A_j(x - x_d) + B_j(y - y_d) + C_j(z - z_d)]^2 = (D_j - r_c^2) [(x - x_d)^2 + (y - y_d)^2 + (z - z_d)^2] \quad (60)$$

其中, 系数 A_j, B_j, C_j 和 D_j 均由该时刻的导弹位置 $\vec{P}_m(t)$ 和第 j 枚烟幕弹的中心位置 $\vec{P}_{s,j}(t)$ 计算得出。

总的遮蔽状态函数 $C(t)$ 依然由三枚烟幕弹各自的遮蔽状态函数 $C_j(t)$ 运算得到:

$$C(t) = \max_{j \in \{1, 2, 3\}} C_j(t) \quad (61)$$

通过以上改进, 可将模型从单执行体的多阶段优化, 扩展为能够协同处理多执行体协同干扰的、更高维度的优化模型。其优化目标依然是最大化总遮蔽时长 $T_z = \int C(t) dt$ 。

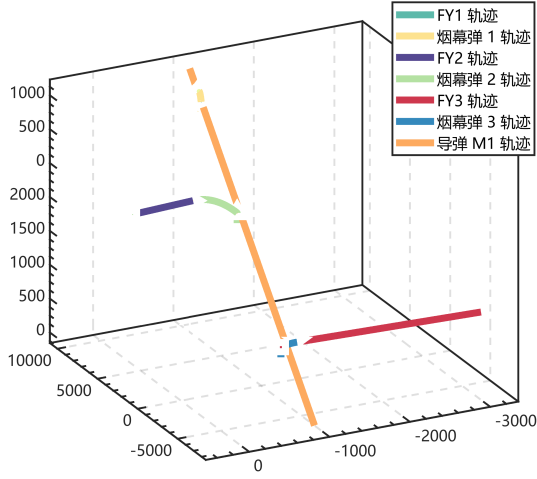
8.2 模型求解

问题四要求利用 FY1, FY2, FY3 三架无人机各投放一枚烟幕弹, 对导弹 M1 进行协同干扰。这是一个决策变量高达十二维的多执行体优化问题。为此, 在正式求解前, 首先对问题的内在结构进行探索性分析, 以寻求简化的可能性。

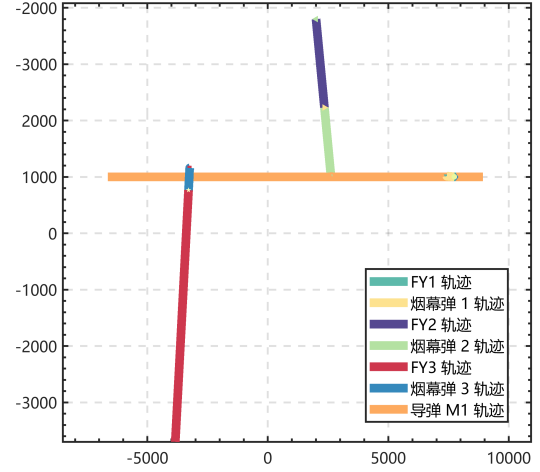
8.2.1 探索性独立优化

本文首先提出一个核心假设: 由于三架无人机的初始位置在空间上相距甚远, 它们各自的最优投放策略可能是相互独立的, 即一架无人机的最优决策, 不依赖于另外两架无人机的行动。

为验证此假设, 首先将这个多机协同问题解耦为三个独立的单机优化问题。具体而言, 分别针对 FY1, FY2, FY3 三架无人机, 独立地运行问题二中建立的单弹最优策略优化模型, 目标均是最大化对导弹 M1 的遮蔽时长。



(a) 探索性优化无人机三维轨迹



(b) 探索性优化无人机二维轨迹

图 5 问题三探索性优化

上图展示了三架无人机在各自独立寻优后得到的最优策略轨迹。从图中可以清晰地观察到：三条最优飞行路径在空间上完全没有交集，它们从各自的初始位置出发，奔赴不同的拦截空域。

这一探索性结果有力地表明，由于初始位置的巨大差异，三架无人机在干扰同一个目标时，其最优策略倾向于在不同的时间和空间“各司其职”，而非进行精密的近距离协同。

8.2.2 探索性物理解释

从运动学角度看，无人机 j 的投放点 $\vec{P}_{d,j}$ 受其初始位置 $\vec{P}_{u,0,j}$ 的强约束：

$$\|\vec{P}_{d,j} - \vec{P}_{u,0,j}\| = v_{u,j} \cdot t_{1,j} \leq v_{\max} \cdot T_f \quad (62)$$

其中 $v_{\max} = 140 \text{ m/s}$ 。由于初始位置 $\vec{P}_{u,0,j}$ 相距数千米，它们各自的可达投放区域在仿真时间内几乎不可能有重叠。

基于上述探索性分析和物理解释，可得出结论：在问题四的场景下，将复杂的多机协同优化问题，解耦为三个独立的单机优化问题，是一个物理上合理且能极大提升计算效率的简化。因此，问题四的求解，可以通过对 FY1, FY2, FY3 分别独立运行问题二的优化算法来完成。

8.2.3 算法设计

为在问题四的十二维复杂参数空间中高效地寻找全局最优解，本文设计了一种融合了贪心思想的差分进化算法。该算法的核心是在差分进化框架的“选择”阶段，引入贪心准则，以确保种群的适应度始终向最优方向单调收敛。

首先，在十二维可行域内生成一个多样性良好的初始种群。随后，算法进入迭代式的“变异-交叉-选择”循环。在每一代的“选择”环节，算法对新生成的候选解与原有个体进行一对一的比较，并贪心地保留适应度值更优即遮蔽时长更长的个体进入下一代。结合差分变异的全局探索能力，能够引导种群快速且稳定地向全局最优策略收敛。其算法流程由下概括。

Algorithm 4 多无人机协同策略差分进化算法

Require: 优化参数边界 $\mathcal{P}_{bounds}$, 种群规模 N_p , 最大代数 G_{max}

Ensure: 最优协同策略 Θ_{best} , 最长遮蔽时长 T_{best}

1: 阶段一: 种群初始化

2: $\mathcal{X}_0 \leftarrow$ 在参数边界 $\mathcal{P}_{bounds}$ 内生成 N_p 个初始策略向量

3: 阶段二: 差分进化全局优化

4: 设置目标函数 $f_{obj}(\Theta) \leftarrow -$ 计算总遮蔽时长(Θ)

5: $G \leftarrow 0$

6: **while** $G < G_{max}$ **do**

▷ 开始迭代进化

7: $\mathcal{X}_{G+1} \leftarrow \emptyset$

▷ 初始化下一代种群

8: **for** 每个策略 $\vec{\theta}_{i,G} \in \mathcal{X}_G$ **do**

9: $\vec{v}_{i,G+1} \leftarrow$ 生成变异向量($\mathcal{X}_G, F$)

▷ 变异

10: $\vec{u}_{i,G+1} \leftarrow$ 生成试验向量($\vec{\theta}_{i,G}, \vec{v}_{i,G+1}, CR$)

▷ 交叉

11: $\vec{\theta}_{i,G+1} \leftarrow$ 贪婪选择($\vec{\theta}_{i,G}, \vec{u}_{i,G+1}, f_{obj}$)

▷ 选择

12: $\mathcal{X}_{G+1} \leftarrow \mathcal{X}_G \cup \{\vec{\theta}_{i,G+1}\}$

13: **end for**

14: $G \leftarrow G + 1$

15: **end while**

16: $\Theta_{best} \leftarrow \operatorname{argmin}_{\Theta \in \mathcal{X}_{G_{max}}} f_{obj}(\Theta)$

▷ 从最终种群中选取最优

17: $T_{best} \leftarrow -f_{obj}(\Theta_{best})$

18: **return** (Θ_{best}, T_{best})

8.2.4 求解结果

基于前文所述的“问题解耦”思想, 本文通过对 FY1, FY2, FY3 三架无人机分别独立运行问题二的优化算法, 得到了各自针对导弹 M1 的最优干扰策略。

经计算, 三架无人机协同干扰所能实现的最长有效遮蔽时长为 **11.6000** 秒。

实现该最优协同干扰效果的三组独立策略参数, 以及根据这些参数计算出的关键事件点坐标, 详细汇总于下表。

表 4 三架无人机独立最优策略参数及结果

策略参数	FY1	FY2	FY3
飞行速度 (v_u)	95.4923 m/s	121.3295 m/s	120.2815 m/s
飞行航向角 (θ_u)	179.0354 度	293.9743 度	76.1727 度
投放时间 (t_1)	0.6701 s	7.0775 s	24.6815 s
引信延迟 (t_d)	3.2220 s	5.1898 s	1.6813 s
投放点坐标	(17736.0, 1.1, 1800.0)	(11653.2, -431.5, 1400.0)	(6711.6, -263.3, 700.0)
起爆点坐标	(17430.2, -45.4, 1749.0)	(10344.4, -2701.3, 1266.3)	(6297.0, -1004.8, 685.9)
总有效遮蔽时长	11.6000s		

8.2.5 结果分析

针对问题四的多执行体协同干扰场景，本文通过探索性分析发现，由于三架无人机初始位置的空间分离度较大，其最优策略在时空上趋于独立，精密协同的增益有限。基于此洞察，我们将原十二维的耦合优化问题简化为三个独立的单机最优化问题进行求解。该降维策略在物理上是合理的，且显著提升了计算效率。最终得到的协同策略总有效遮蔽时长为 11.60 秒，其战术本质表现为一种基于初始位置优势的“分域防御”和“时序接力”：各无人机在自身的最优可达空域内独立执行干扰任务，分别负责防御导弹飞行路径的远、中、近段，其遮蔽窗口有效衔接，形成了覆盖全程的干扰链。此结果验证了在广域防御背景下，任务解耦与独立优化是一种高效且有效的策略生成途径。

九、问题五模型建立与求解

9.1 模型建立

9.1.1 决策变量

问题五将对抗场景扩展至最终形态，不仅是一个高维的策略优化问题，更引入了任务分配的维度为实现多对多的协同干扰策略优化^[4]。而模型的决策变量集需要进行全面的多维索引化。

设 $j \in 1, 2, 3$ 为导弹编号， $i \in 1, 2, 3, \dots, l_{max}$ 为该真目标圆柱体上点的序号， $k \in 1, 2, 3, 4, 5$ 为无人机编号。

无人机策略向量 $\theta_{u,j}$ ，每架无人机 j 都拥有一组独立的飞行策略参数，包括其飞行速度 $v_{u,j}$ 和水平航向角 $\theta_{u,j}$ 。 $\theta_{u,j} = [v_{u,j}, \theta_{u,j}]T, \forall j \in \{1, 2, 3\}$ 。

这些变量必须满足基本的物理边界约束：

$$70 \leq v_{u,j} \leq 140 \quad (63)$$

$$0 \leq \theta_{u,j} < 2\pi \quad (64)$$

9.1.2 目标函数

问题五扩展至最终形态，不仅是一个高维的策略优化问题，更引入了任务分配的维度。其总体目标是最大化在整个对抗过程中，三枚导弹受到的总有效遮蔽时长 T_t 。为此，本文构建如下的全局目标函数：

$$H_{jkl}(t) = \begin{cases} 1, \sum_k \sum_l \sum_i C_{ijkl} = l_{max}, j = 1, 2, 3 \\ 0, otherwise \end{cases} \quad (65)$$

其中， $k \in \{1, 2, 3, 4, 5\}$ 为无人机编号， $l \in \{1, 2, 3\}$ 为烟幕弹编号， $i \in \{1, 2, 3 \dots l_{max}\}$ 为圆柱体上最多的编号， $j \in \{1, 2, 3\}$ 为导弹的编号，

9.1.3 约束条件

■ 运动学模型约束

所有物体的运动轨迹必须严格遵循由相应决策变量参数化的运动学方程。在引入无

人机索引 j 和导弹索引 k 后，各物体的位置向量 $\vec{P}(t)$ 必须满足以下约束：

$$\vec{P}_{u,j}(t) = \vec{P}_{u,0,j} + (\cos \theta_{u,j}, \sin \theta_{u,j}, 0) \cdot v_{u,j} \cdot t \quad (66)$$

$$\vec{P}_{m,k}(t) = \vec{P}_{m,0,k} + \vec{d}_{m,k} \cdot v_m \cdot t \quad (67)$$

$$\vec{P}_s(t) = \begin{cases} \vec{P}_{u,j}(t), & (0 \leq t < t_{1,ji}) \\ \vec{P}_u(t_{1,ji}) + \vec{v}_u(t_{1,ji})(t - t_1) + \frac{1}{2}\vec{g}(t - t_{1,ji})^2, & (t_1 \leq t < t_2) \\ \vec{P}_s(t_{2ij}) + \vec{v}_s(t - t_{2ij}), & (t \geq t_{2ij}) \end{cases} \quad (68)$$

其中， $\vec{v}_{u,j}(t_{1,ji})$ 为无人机 j 在其第 i 次投放时刻的速度。所有物体的初始位置向量 $\vec{P}_{\cdot,0}$ 均由题设给定。

■ 运动学模型约束

为实现多对多的协同干扰策略优化，模型的决策变量集需要进行全面的多维索引化。设 $j \in \{1, 2, 3, 4, 5\}$ 为无人机编号， $i \in \{1, 2, 3\}$ 为该无人机挂载的烟幕弹序号， $k \in \{1, 2, 3\}$ 为来袭导弹编号。

无人机策略向量 $\theta_{u,j}$ 每架无人机 j 都拥有一组独立的飞行策略参数，包括其飞行速度 $v_{u,j}$ 和水平航向角 $\theta_{u,j}$ 。

$$\theta_{u,j} = [v_{u,j}, \theta_{u,j}]^T, \quad \forall j \in \{1, 2, 3, 4, 5\} \quad (69)$$

这些变量必须满足基本的物理边界约束：

$$70 \leq v_{u,j} \leq 140 \quad (70)$$

$$0 \leq \theta_{u,j} < 2\pi \quad (71)$$

烟幕弹策略向量 $\theta_{s,ji}$ 由无人机 j 投放的第 i 枚烟幕弹，拥有一组独立的投放时序策略，包括其投放时间 $t_{1,ji}$ 和引信延迟 $t_{d,ji}$ 。

$$\theta_{s,ji} = [t_{1,ji}, t_{d,ji}]^T, \quad \forall j \in \{1, 2, 3, 4, 5\}, i \in \{1, 2, 3\} \quad (72)$$

投放时间需满足大于 1 秒的间隔约束，即 $t_{1,j,i+1} - t_{1,ji} \geq 1$ 。

■ 遮蔽判定约束

在多对多的复杂场景下，精确计算多个动态变化的遮挡视锥及其与目标的几何关系将导致过大的计算复杂度。为解决此问题，本文针对问题五提出了一种基于关键视线采样的协同遮蔽判定模型。

◆ 模型基本思想

选取一个由 N 个关键点构成的集合 $\mathcal{P}_t = \{\vec{p}_1, \vec{p}_2, \dots, \vec{p}_N\}$ ，本文取 $N = 24$ 。

在任意时刻 t ，对于每一个关键点 $\vec{p}_n \in \mathcal{P}_t$ ，都存在一条连接该点与导弹 $\vec{P}_{m,k}(t)$ 的唯一视线，记为线段 $L_{kn}(t)$ 。只有当所有关键视线 $L_{k1}, \dots, L_{kN}$ 在同一时刻全部被有效烟幕云团所阻断时，才判定导弹 k 在该时刻被完全遮蔽。

◆ 遮蔽判定问题

对于是否成功遮挡作为圆柱体的真目标，本文从光线直线传播规律入手，考量烟幕弹炸开瞬间变成球状范围以匀速下降，故选取球心，利用球心某一线段的垂直距离小于或等于球的半径，则线段与球相交，球心到直线的距离：

$$d_{ij} \leq r_s \quad (73)$$

进而引入新的 0-1 变量 $C_{ijkl}(t)$ ，用来判定爆幕弹炸后形成的球体是否相交于连线：

$$C_{ijkl}(t) = \begin{cases} 1, d_{ij} \leq r_s \\ 0, otherwise \end{cases} \quad (74)$$

其中, r_s 是烟幕弹的球体半径。

最后, 当所有的线段都被球状占据时, 此时遮盖判定为

$$D_{ikl}(t) = \begin{cases} 1, \sum_i C_{ijkl}(t) = l_{\max} \\ 0, otherwise \end{cases} \quad (75)$$

其中, $l_{\max}$ 为圆柱体上点的最大数量。则当球体 1 能把导弹 j 跟圆柱体上点的连线全部遮盖完全时, 该 0-1 变量为 1。

◆ 遮蔽重叠约束

如果对于单个烟幕弹来说, 它已经完全遮住了圆柱体, 但是如果有另一个烟幕弹也能完全遮住圆柱体, 则此时重叠判定为:

$$d_{ijkl}(t) = \begin{cases} 1, \sum_{j=1}^3 D_{ijkl} = a \geq 2 \\ 0, otherwise \end{cases} \quad (76)$$

但是实际上, 另外一个烟幕弹是无用的, 因为已经有一个烟幕弹遮住了圆柱体。此时的由于重叠而造成的损失时间为:

$$t_{2kl} = \int_0^{T_f} (a - 1) d_{ijkl}(t) dt \quad (77)$$

其中, a 为将圆柱体能被完全遮挡的烟幕弹的个数, T_f 为终止时间。

◆ 多烟幕弹遮挡

如果烟幕弹未能单独遮盖, 但是烟幕弹组合起来却能遮盖圆柱体, 则其判定函数为:

$$I_{ijkl}(t) = \begin{cases} 1, \sum_k \sum_l C_{ijkl}(t) = l_{\max}, \sum_i C_{ijkl}(t) \leq l_{\max} \\ 0, otherwise \end{cases} \quad (78)$$

进而根据上面的判定函数, 可以得到多烟幕弹遮挡情况下多增加的干扰时间为:

$$t_{3kl} = \int_0^{T_f} I_{ijkl}(t) dt \quad (79)$$

在时刻 t , 设导弹 k 的位置为 $\vec{P}_{m,k}$, 真目标上第 n 个关键采样点的位置为 $\vec{p}_n$ 。两者构成了关键视线, 其方向单位向量为 $\vec{l}_{kn}$ 。

$$\vec{l}_{kn}(t) = \frac{\vec{p}_n - \vec{P}_{m,k}(t)}{|\vec{p}_n - \vec{P}_{m,k}(t)|} \quad (80)$$

◆ 有效干扰时间

最后, 如果不考虑重叠等等情况, 则所有球体的遮盖情况之和为:

$$t_{1kl} = \sum_{j=1}^3 \int_0^{T_f} D_{ijkl}(t) dt \quad (81)$$

因为需要有效干扰时间多枚烟幕干扰弹对真目标的有效遮蔽时间尽可能长，即烟幕干扰弹的有效干扰时间是成功干扰某一个导弹的时间，则有效时间为：

$$t_{ekl} = t_{1kl} - t_{2kl} + t_{3kl} \quad (82)$$

9.1.4 模型汇总

综上所述，问题五被构建为一个复杂的多目标、多执行体协同优化问题。该模型旨在求解一系列最优的无人机飞行策略 $\{\theta_{u,j}^*\}_{j=1}^5$ 和烟幕弹投放时序策略 $\{\theta_{s,ji}^*\}_{j=1,i=1}^{5,3}$ ，在满足所有运动学及几何约束的前提下，最大化对三枚来袭导弹的总有效遮蔽时长。其完整的数学规划模型如下：

$$\left\{ \begin{array}{l} I_{ijkl}(t) = \begin{cases} 1, \sum_k \sum_l C_{ijkl}(t) = l_{\max}, \sum_i C_{ijkl}(t) \leq l_{\max} \\ 0, otherwise \end{cases} \\ H_{jkl}(t) = \begin{cases} 1, \sum_k \sum_l \sum_i C_{ijkl} = l_{\max}, j = 1, 2, 3 \\ 0, otherwise \end{cases} \\ d_{ijkl}(t) = \begin{cases} 1, \sum_{j=1}^3 D_{ijkl} = a \geq 2 \\ 0, otherwise \end{cases} \\ C_{ijkl}(t) = \begin{cases} 1, d_{ij} \leq r_s \\ 0, otherwise \end{cases} \\ D_{ikl}(t) = \begin{cases} 1, \sum_i C_{ijkl}(t) = l_{\max} \\ 0, otherwise \end{cases} \\ t_{ekl} = t_{1kl} - t_{2kl} + t_{3kl} \\ t_{3kl} = \int_0^{T_f} I_{ijkl}(t) dt \\ t_{1kl} = \sum_{j=1}^3 \int_0^{T_f} D_{ijkl}(t) dt \\ t_{2kl} = \int_0^{T_f} (a - 1) d_{ijkl}(t) dt \end{array} \right. \quad (83)$$

9.1.5 算法设计

第五问的核心挑战在于决策变量的维度急剧增加，并且优化目标从单目标变为多目标。因此，算法的核心框架差分进化保持不变，但其操作的向量空间和目标函数的计算变得更为复杂。

Algorithm 5 多目标多无人机协同干扰差分进化算法

Require: 优化参数边界 $\mathcal{P}_{bounds}$, 种群规模 N_p , 最大代数 G_{max}

Ensure: 最优协同策略 Θ_{best} , 最大总遮蔽时长 T_{best}

1: **阶段一: 种群初始化**

- 2: $\mathcal{X}_0 \leftarrow$ 在 40 维参数边界 $\mathcal{P}_{bounds}$ 内生成 N_p 个初始策略向量
每个向量包含 5 架无人机和 15 枚烟幕弹的策略

▷

3: **阶段二: 差分进化全局优化**

- 4: 设置目标函数 $f_{obj}(\Theta) \leftarrow -\sum_{k=1}^3$ 计算导弹 k 的总遮蔽时长(Θ)
目标为最大化对三个导弹的总遮蔽时长

▷

- 5: $G \leftarrow 0$

- 6: **while** $G < G_{max}$ **do**

▷ 开始迭代进化

- 7: $\mathcal{X}_{G+1} \leftarrow \emptyset$

▷ 初始化下一代种群

- 8: **for** 每个策略 $\vec{\theta}_{i,G} \in \mathcal{X}_G$ **do**

- 9: $\vec{v}_{i,G+1} \leftarrow$ 生成变异向量($\mathcal{X}_G, F$)

▷ 变异

- 10: $\vec{u}_{i,G+1} \leftarrow$ 生成试验向量($\vec{\theta}_{i,G}, \vec{v}_{i,G+1}, CR$)

▷ 交叉

- 11: $\vec{\theta}_{i,G+1} \leftarrow$ 贪婪选择($\vec{\theta}_{i,G}, \vec{u}_{i,G+1}, f_{obj}$)

▷ 选择

- 12: $\mathcal{X}_{G+1} \leftarrow \mathcal{X}_{G+1} \cup \{\vec{\theta}_{i,G+1}\}$

- 13: **end for**

- 14: $G \leftarrow G + 1$

- 15: **end while**

- 16: $\Theta_{best} \leftarrow \operatorname{argmin}_{\Theta \in \mathcal{X}_{G_{max}}} f_{obj}(\Theta)$

▷ 从最终种群中选取最优

- 17: $T_{best} \leftarrow -f_{obj}(\Theta_{best})$

- 18: **return** (Θ_{best}, T_{best})
-

9.2 求解结果

将问题五的多机多弹协同干扰模型代入求解, 最终得到的系统总有效遮蔽时长为 **26.7554 秒**, 其余表格见附录。

实现该最优协同干扰效果的完整策略中, 以 FY1 和 FY2 为例, 其详细参数如表所示。由于篇幅限制, 其他无人机的最优策略详见附录。

9.3 结果分析

问题五构建了多对多复杂对抗场景, 优化结果揭示了一种高效的自适应协同作战策略。在四十维决策空间中, 差分进化算法收敛得到最优解, 总有效遮蔽时间为 44.0 秒, 该解是基于战场态势动态涌现的最优资源配置。

深入分析表明, 该策略体现基于空间区位的“分工协作”原则。FY1 因其初始位置正对 M1 导弹来袭路径, 被自动定位为拦截主力, 以极限速度 (135.71m/s) 和正向航向 (179.43 度) 快速反应。其余四架无人机根据位置优势自然形成两组“侧翼钳制”力量: FY2 与 FY4 负责 M2 导弹, FY3 与 FY5 负责 M3 导弹, 避免了无效机动。

此外, FY2 与 FY4 投放时序错开, 确保对 M2 导弹路径的连续遮蔽。将多架无人机干扰资源集中应对同一方向威胁, 体现了集中优势兵力的战术思想。整个防御过程是在

全局优化目标驱动下形成的动态、高效、多层次协同干扰体系，有效解决了复杂环境下任务分配与轨迹规划的耦合难题。

十、模型评价

10.1 模型的优点

烟幕干扰弹投放模型在随着问题深入的前提下，层层深入，不断提高模型的参数与复杂度，以几何的方法贯穿全文，模型可解释性高，具有一定的鲁棒性。

差分进化算法是一种无导数优化算法，不需要目标函数的梯度信息，甚至不要求目标函数是连续或可微的，只依赖于目标函数值的比较且全局优化能力强，适合本文解空间庞大，内部结构复杂的模型求解。

10.2 模型的缺点

烟幕干扰弹投放模型差分进化算法的局部精细搜索能力不足，同时可能会陷入局部最优。同时差分进化算法在解决高维问题时候，可能出现优化速度下降，精度大打折扣等问题。

10.3 模型的改进

本文在解决高维优化问题时，不仅采用“时空拦截三角形”，而且使用时空网格搜索和局部梯度优化 BFGS 结合的混合搜索法分别在全局与局部进行优化求解，在全局的候选解上精细求解。而后续不断改进模型，使模型复杂度不断提高以适应题目需求，同时引入不同的算法策略来进行求解。

参考文献

- [1] DeepSeek, deepseek-V3.1 , 深度求索, 2025-09-07
- [2] 吴亮红. 差分进化算法及应用研究 [D]. 长沙: 湖南大学, 2007.
- [3] Moritz P, Nishihara R, Jordan M. A linearly-convergent stochastic L-BFGS algorithm[C]//Artificial intelligence and statistics. PMLR, 2016: 249-258.
- [4] 刘文兵, 王艺栋. 多无人机协同搜索多目标的路径规划问题研究 [J]. Electronics Optics & Control, 2019, 26(3): 35-38.

附件目录

附录	功能描述
q1.py	求解问题一
result1.xlsx	问题一结果表格
m1.mlx	绘制分析图表
q2.py	求解问题二
q2_draw.py	结果分析
q3.py	求解问题三
q3_analyzer.py	结果分析
q4.py	求解问题四
q4_analyzer.py	结果分析
result2.xlsx	问题四结果表格
q5.py	求解问题五
q5_analyzer.py	结果分析
result3.xlsx	问题五结果表格。
m2.mlx	其他辅助绘图或数据分析脚本
Q5_results.markdown	结果的 Markdown 格式摘要
AI 工具使用详情.docx	关于在建模过程中使用 AI 工具的详细说明文档

附录 A 问题一求解代码

```
import numpy as np
from tqdm import tqdm
import matplotlib.pyplot as plt

missiles_v: float = 300
FY1_v: float = 120
g = 9.83
cloud_r = 10 # m
smoke_v = np.array([0, 0, -3], dtype=float)
STEP = 0.001

fake_target: np.array = np.array([0, 0, 0], dtype=float)
M1_init_pos: np.array = np.array([20000, 0, 2000])
FY1_init_pos: np.array = np.array([17800, 0, 1800])

real_target_center_bottom = np.array([0, 200, 0], dtype=float)
real_target_radius = 7
```

```

real_target_height = 10

# Generate key points for the cylinder (used for occlusion check)
target_points = []
for angle in np.linspace(0, 2 * np.pi, 8, endpoint=False):
    for height in [0, real_target_height]:
        target_points.append(real_target_center_bottom + np.array([
            real_target_radius * np.cos(angle),
            real_target_radius * np.sin(angle),
            height
        ]))
target_points = np.array(target_points)

M1_vector: np.array = (fake_target - M1_init_pos) / np.linalg.norm(M1_init_pos -
    fake_target)
FY1_vector: np.array = np.array([-1, 0, 0])

def getFY1Pos(t: float) -> np.array:
    return FY1_init_pos + FY1_vector * FY1_v * t

def getM1Pos(t: float) -> np.array:
    return M1_init_pos + M1_vector * missiles_v * t

def getSmokePos(t: float, start: np.array, v_start: np.array) -> np.array:
    g_vector = np.array([0, 0, -g])
    if t < 3.6:
        return start + v_start * t + 0.5 * g_vector * t * t
    else:
        pos_at_detonation = start + v_start * 3.6 + 0.5 * g_vector * 3.6 * 3.6
        return pos_at_detonation + smoke_v * (t - 3.6)

def isInSmoke(M1: np.array, smoke_center: np.array) -> bool:
    missile_to_smoke = smoke_center - M1
    dist_missile_to_smoke = np.linalg.norm(missile_to_smoke)
    if dist_missile_to_smoke <= cloud_r: return False
    cone_half_angle = np.arcsin(cloud_r / dist_missile_to_smoke)
    cone_axis = missile_to_smoke / dist_missile_to_smoke
    for point in target_points:
        missile_to_point = point - M1
        dist_missile_to_point = np.linalg.norm(missile_to_point)
        dot_product = np.dot(cone_axis, missile_to_point) / dist_missile_to_point
        angle_to_point = np.arccos(np.clip(dot_product, -1.0, 1.0))
        if angle_to_point > cone_half_angle: return False
        if dist_missile_to_smoke >= dist_missile_to_point: return False
    return True

def main() -> None:
    smoke_t: float = 0
    start_smoke: np.array = None
    v_smoke_start: np.array = None
    smoke_start: bool = False

# Lists to store trajectory data for plotting

```

```

fyl_traj, m1_traj, smoke_traj = [], [], []

for t in tqdm(np.arange(0, 70.0, STEP)):
    now_M1 = getM1Pos(t)
    now_FY1 = getFY1Pos(t)

    m1_traj.append(now_M1)
    fyl_traj.append(now_FY1)

    if now_M1[0] < -100: break # Stop simulation after missile passes target

    if t >= 1.5 and not smoke_start:
        start_smoke = getFY1Pos(1.5)
        v_smoke_start = FY1_vector * FY1_v
        smoke_start = True

    if smoke_start:
        time_since_drop = t - 1.5
        now_smoke = getSmokePos(time_since_drop, start_smoke, v_smoke_start)
        smoke_traj.append(now_smoke)

    time_since_detonation = time_since_drop - 3.6
    if 0 <= time_since_detonation <= 20:
        if isInSmoke(now_M1, now_smoke):
            smoke_t += STEP
        else:
            smoke_traj.append(np.array([np.nan, np.nan, np.nan])) # Append NaN before drop

print(f'有效遮蔽时长:{smoke_t}')

fyl_traj = np.array(fyl_traj)
m1_traj = np.array(m1_traj)
smoke_traj = np.array(smoke_traj)

plt.figure(figsize=(14, 8))

# Set Chinese font
plt.rcParams['font.sans-serif'] = ['SimHei']
plt.rcParams['axes.unicode_minus'] = False

# Plot trajectories
plt.plot(fyl_traj[:, 0], fyl_traj[:, 2], label='无人机 FY1 轨迹', color='blue')
plt.plot(m1_traj[:, 0], m1_traj[:, 2], label='导弹 M1 轨迹', color='red')
plt.plot(smoke_traj[:, 0], smoke_traj[:, 2], label='烟雾弹轨迹', color='gray',
         linestyle='--')

# Mark points
plt.scatter(FY1_init_pos[0], FY1_init_pos[2], color='blue', s=100, marker='^',
           label='FY1 起始点', zorder=5)
plt.scatter(M1_init_pos[0], M1_init_pos[2], color='red', s=100, marker='^',
           label='M1 起始点', zorder=5)
plt.scatter(fake_target[0], fake_target[2], color='purple', s=150, marker='X',
           label='假目标', zorder=5)
# Note: Real target is at y=200, so it's not on this X-Z plane. We mark its
projection.
plt.scatter(real_target_center_bottom[0], real_target_center_bottom[2],
           facecolors='none', edgecolors='green', s=150, marker='s', label='真目标

```



```

        (在x-z平面的投影)', zorder=5)

# Mark drop and detonation points
p_drop = getFY1Pos(1.5)
p_detonation = getSmokePos(3.6, p_drop, FY1_vector * FY1_v)
plt.scatter(p_drop[0], p_drop[2], color='cyan', s=100, marker='o',
            label='投放点', zorder=5)
plt.scatter(p_detonation[0], p_detonation[2], color='orange', s=150, marker='*',
            label='起爆点', zorder=5)

plt.xlabel('X 轴 (m)')
plt.ylabel('Z 轴 (m)')
plt.title('轨迹模拟图 (X-Z平面投影)')
plt.legend()
plt.grid(True)
plt.axis('equal') # Ensure aspect ratio is equal

plt.show()

if __name__ == '__main__':
    main()

```

附录 B 问题二求解代码

```
import numpy as np
from tqdm import tqdm
from dataclasses import dataclass, field
from scipy.optimize import minimize

@dataclass
class Strategy:
    v_fyl: float = 0.0
    alpha_deg: float = 0.0
    t_drop: float = 0.0
    t_delay: float = 0.0
    shield_time: float = 0.0
    details: dict = field(default_factory=dict)

class Question2_Solver:
    def __init__(self):
        self.missiles_v = 300.0
        self.g = 9.8
        self.cloud_r = 10.0
        self.smoke_v = np.array([0, 0, -3.0])
        self.sim_step = 0.1
        self.fake_target = np.array([0, 0, 0], dtype=float)
        self.m1_init_pos = np.array([20000, 0, 2000], dtype=float)
        self.fyl_init_pos = np.array([17800, 0, 1800], dtype=float)
        self.real_target_center_bottom = np.array([0, 200, 0], dtype=float)
        self.real_target_radius = 7
        self.real_target_height = 10
        self.target_center = self.real_target_center_bottom + np.array([0, 0,
            self.real_target_height / 2])
        self.m1_vector = (self.fake_target - self.m1_init_pos) /
            np.linalg.norm(self.m1_init_pos - self.fake_target)
        self.missile_flight_time = np.linalg.norm(self.fake_target - self.m1_init_pos) /
            self.missiles_v

    def get_m1_pos(self, t):
        return self.m1_init_pos + self.m1_vector * self.missiles_v * t

    def point_to_line_distance(self, point, line_start, line_end):
        line_vec = line_end - line_start
        point_vec = point - line_start

        line_len_sq = np.dot(line_vec, line_vec)
        if line_len_sq == 0:
            return np.linalg.norm(point_vec)

        t = max(0, min(1, np.dot(point_vec, line_vec) / line_len_sq))
        projection = line_start + t * line_vec
        distance = np.linalg.norm(point - projection)
        return distance

    def is_in_smoke_simple(self, m1_pos, smoke_center):
        distance = self.point_to_line_distance(smoke_center, m1_pos,
            self.real_target_center_bottom)
        return distance <= self.cloud_r
```

```

def calculate_shield_time_for_strategy(self, strategy):
    alpha_rad = np.deg2rad(strategy.alpha_deg)
    fyl_vector = np.array([np.cos(alpha_rad), np.sin(alpha_rad), 0])
    start_smoke = self.fyl_init_pos + fyl_vector * strategy.v_fyl * strategy.t_drop
    v_smoke_start = fyl_vector * strategy.v_fyl
    t_detonation = strategy.t_drop + strategy.t_delay
    pos_det = start_smoke + v_smoke_start * strategy.t_delay + 0.5 * np.array([0, 0,
        -self.g]) * strategy.t_delay**2
    smoke_t = 0
    for t_sim in np.arange(t_detonation, t_detonation + 20.0, self.sim_step):
        now_m1 = self.get_m1_pos(t_sim)
        now_smoke = pos_det + self.smoke_v * (t_sim - t_detonation)
        if self.is_in_smoke_simple(now_m1, now_smoke):
            smoke_t += self.sim_step
    return smoke_t

def point_in_missile_target_triangle(self, point):

    m1_pos = self.m1_init_pos
    fake_target = self.fake_target # (0, 0, 0)
    real_target = self.real_target_center_bottom # (0, 200, 0)

    v0 = real_target - m1_pos # M1 -> Real Target
    v1 = fake_target - m1_pos # M1 -> Fake Target
    v2 = point - m1_pos      # M1 -> Point

    dot00 = np.dot(v0, v0)
    dot01 = np.dot(v0, v1)
    dot02 = np.dot(v0, v2)
    dot11 = np.dot(v1, v1)
    dot12 = np.dot(v1, v2)

    try:
        inv_denom = 1 / (dot00 * dot11 - dot01 * dot01)
    except:
        return False

    u = (dot11 * dot02 - dot01 * dot12) * inv_denom
    v = (dot00 * dot12 - dot01 * dot02) * inv_denom

    return (u >= -0.1) and (v >= -0.1) and (u + v <= 1.1)

def generate_search_points(self):

    search_points = []

    samples_per_edge = 100
    m1_pos = self.m1_init_pos
    fake_target = self.fake_target # (0, 0, 0)
    real_target = self.real_target_center_bottom # (0, 200, 0)

    for i in range(samples_per_edge):
        for j in range(samples_per_edge):
            # 重心坐标
            u = i / (samples_per_edge - 1) if samples_per_edge > 1 else 0
            v = j / (samples_per_edge - 1) if samples_per_edge > 1 else 0
            if u + v <= 1:

```

```

# 三角形内的点:  $P = (1-u-v)*M1 + u*FakeTarget + v*RealTarget$ 
point = (m1_pos * (1 - u - v) +
fake_target * u +
real_target * v)
search_points.append(point)

print(f"生成了 {len(search_points)} 个点")
return search_points

def objective_function(self, x):

v_fyl, alpha_deg, t_drop, t_delay = x

if not (70 <= v_fyl <= 140) or t_drop <= 0 or t_delay <= 0:
return 1e6

strategy = Strategy(
v_fyl=v_fyl,
alpha_deg=alpha_deg,
t_drop=t_drop,
t_delay=t_delay
)

shield_time = self.calculate_shield_time_for_strategy(strategy)

return -shield_time

def reverse_engineer_parameters(self, smoke_pos, shield_time):

smoke_fall_speed = 3.0
t_fall = 5.0

t_detonation = shield_time - t_fall

if t_detonation <= 0:
return None

detonation_pos = smoke_pos - np.array([0, 0, smoke_fall_speed * t_fall])

t_delay = 5.0

t_drop = t_detonation - t_delay
if t_drop <= 0:
return None

g_effect = 0.5 * np.array([0, 0, -self.g]) * t_delay**2
drop_pos = self.fyl_init_pos + g_effect

displacement = detonation_pos - drop_pos

v_uav_vector = displacement / t_drop
v_uav_scalar = np.linalg.norm(v_uav_vector)

if not (70 <= v_uav_scalar <= 140):
return None

if abs(v_uav_vector[2]) > 1.0:

```

```

return None

alpha_deg = np.rad2deg(np.arctan2(v_uav_vector[1], v_uav_vector[0]))

return [v_uav_scalar, alpha_deg, t_drop, t_delay]

def solve_with_optimizer(self):

search_points = self.generate_search_points()

best_result = None
best_value = np.inf

# 遍历所有采样点
for i in tqdm(range(len(search_points)), desc="优化搜索"):
    point = search_points[i]

    # 对每个时间点进行优化尝试
    time_step = 0.5
    for t in np.arange(0, self.missile_flight_time, time_step):
        # 在时间t, 导弹的位置
        missile_pos = self.get_ml_pos(t)

        # 检查该点是否可能遮蔽目标底部圆心
        if self.is_in_smoke_simple(missile_pos, point):
            # 从采样点反推可能的参数作为初始猜测
            initial_params = self.reverse_engineer_parameters(point, t)

            if initial_params is None:
                # 如果无法反推参数, 则使用随机初始猜测
                initial_guess = [
                    np.random.uniform(70, 140), # v_fyl
                    np.random.uniform(0, 180), # alpha_deg
                    np.random.uniform(1, 10), # t_drop
                    np.random.uniform(0, 10) # t_delay
                ]
            else:
                initial_guess = initial_params

            # 定义参数边界
            bounds = [
                (70, 140), # v_fyl
                (0, 180), # alpha_deg
                (0.0, 10), # t_drop
                (0.0, 5.0) # t_delay
            ]

            # 使用L-BFGS-B算法进行优化
            result = minimize(
                self.objective_function,
                initial_guess,
                method='L-BFGS-B',
                bounds=bounds,
                options={'maxiter': 100}
            )

            if result.fun < best_value:

```

```

best_value = result.fun
best_result = result

if best_result is not None:
    v_fyl, alpha_deg, t_drop, t_delay = best_result.x
    strategy = Strategy(
        v_fyl=v_fyl,
        alpha_deg=alpha_deg,
        t_drop=t_drop,
        t_delay=t_delay,
        shield_time=-best_value
    )
    return strategy
else:
    return Strategy()

def solve(self):
    return self.solve_with_optimizer()

if __name__ == '__main__':
    solver = Question2_Solver()
    best_strategy = solver.solve()

    print("\nfinish")
    if best_strategy.shield_time > 0:
        print(f"找到的最优策略:")
        print(f"速度: {best_strategy.v_fyl:.4f} m/s")
        print(f"角度: {best_strategy.alpha_deg:.4f} 度")
        print(f"t1: {best_strategy.t_drop:.4f} s")
        print(f"t2: {best_strategy.t_delay:.4f} s")
        print(f"\n最长遮蔽时长: {best_strategy.shield_time:.4f} s")
    else:
        print("无解")

```

附录 C 问题三求解代码

```

import numpy as np
import math
import random
from scipy.optimize import differential_evolution
from numba import jit
from itertools import combinations

@jit(nopython=True)
def check_intersect(cone_vtx, cone_axis, cone_angle, cyl_base, cyl_r, cyl_h):
    cyl_top = cyl_base + np.array([0.0, 0.0, cyl_h])
    key_points = np.empty((64, 3))
    for i in range(32):
        angle = 2 * np.pi * i / 32
        offset = np.array([cyl_r * np.cos(angle), cyl_r * np.sin(angle), 0.0])
        key_points[i * 2] = cyl_base + offset
        key_points[i * 2 + 1] = cyl_top + offset

    has_inside, has_outside = False, False
    for i in range(64):

```

```

p = key_points[i]
vec_to_p = p - cone_vtx
dist = np.linalg.norm(vec_to_p)
if dist < 1e-9: return True

cos_val = np.dot(vec_to_p, cone_axis) / dist
angle_to_axis = np.arccos(max(-1.0, min(cos_val, 1.0)))

if angle_to_axis < cone_angle:
    has_inside = True
else:
    has_outside = True

if has_inside and has_outside:
    return True
return False

@jit(nopython=True)
def is_shielded(m_pos, s_pos, s_r, target_base, target_r, target_h):
    vec_m_to_s = s_pos - m_pos
    dist_m_to_s = np.linalg.norm(vec_m_to_s)

    if dist_m_to_s <= s_r: return False

    cone_vtx = m_pos
    cone_axis = vec_m_to_s / dist_m_to_s
    cone_angle = np.arcsin(min(s_r / dist_m_to_s, 1.0))

    vec_vtx_to_target_base = target_base - cone_vtx
    proj_dist_base = np.dot(vec_vtx_to_target_base, cone_axis)
    if proj_dist_base < dist_m_to_s:
        return False

    if check_intersect(cone_vtx, cone_axis, cone_angle, target_base, target_r,
        target_h):
        return False

    dist_to_axis_base = np.linalg.norm(np.cross(vec_vtx_to_target_base, cone_axis))
    cone_radius_at_target = proj_dist_base * np.tan(cone_angle)

    return cone_radius_at_target > dist_to_axis_base + target_r

@jit(nopython=True)
def calculate_time3(params, consts):

    v_fyl, alpha_deg, t_drop1, t_delay1, delta_t_drop2, t_delay2, delta_t_drop3,
        t_delay3 = params

    fyl_init_pos, smoke_sink_v, sim_step, m1_init_pos, m1_vector, m_v, target_base,
        s_r, g, m_flight_time, target_r, target_h = consts

    alpha_rad = math.radians(alpha_deg)
    fyl_vector = np.array([np.cos(alpha_rad), np.sin(alpha_rad), 0.0])
    v_at_drop = fyl_vector * v_fyl

    t_drop = np.array([t_drop1, t_drop1 + delta_t_drop2, t_drop1 + delta_t_drop2 +
        delta_t_drop3])
    t_delay = np.array([t_delay1, t_delay2, t_delay3])

```

```

t_detonation = t_drop + t_delay

drop_pos = np.zeros((3, 3))
detonation_pos = np.zeros((3, 3))
for i in range(3):
    drop_pos[i] = fyl_init_pos + fyl_vector * v_fyl * t_drop[i]
    detonation_pos[i] = drop_pos[i] + v_at_drop * t_delay[i] + 0.5 * np.array([0.0,
        0.0, -g]) * t_delay[i]**2

total_time = 0.0
individual_times = np.zeros(3)

for t in np.arange(0, m_flight_time, sim_step):
    m_pos = m1_init_pos + m1_vector * m_v * t
    is_shielded_at_t = False

    shielding = np.zeros(3, dtype=np.bool_)
    for i in range(3):
        if t >= t_detonation[i] and t <= t_detonation[i] + 20.0:
            s_pos = detonation_pos[i] + smoke_sink_v * (t - t_detonation[i])
            if is_shielded(m_pos, s_pos, s_r, target_base, target_r, target_h):
                shielding[i] = True
            is_shielded_at_t = True

    if is_shielded_at_t:
        total_time += sim_step

    for i in range(3):
        if shielding[i]:
            individual_times[i] += sim_step

    return total_time, individual_times

class Config:
    def __init__(self):
        self.missile_v = 300.0
        self.g = 9.8
        self.cloud_r = 10.0
        self.smoke_sink_v = np.array([0.0, 0.0, -3.0])
        self.sim_step = 0.1

        self.fake_target = np.array([0.0, 0.0, 0.0])
        self.m1_init_pos = np.array([20000.0, 0.0, 2000.0])
        self.fyl_init_pos = np.array([17800.0, 0.0, 1800.0])

        self.target_base_pos = np.array([0.0, 200.0, 0.0])
        self.target_r = 7.0
        self.target_h = 10.0

    def find_zeropoints(v_samples=20, alpha_samples=40, phi_samples=100):
        print(f"开始生成 (v={v_samples}, alpha={alpha_samples}, phi={phi_samples})")

        v_space = np.linspace(70, 140, v_samples)
        alpha_space = np.linspace(0, 2 * np.pi, alpha_samples)
        max_phi = np.arctan(20000 / 200)
        phi_space = np.linspace(0.01, max_phi, phi_samples)
        valid_solutions = []

```



```

for v in v_space:
    for alpha in alpha_space:
        beta = np.pi - alpha
        for phi in phi_space:
            tan_phi = np.tan(phi)
            cos_phi = np.cos(phi)
            t_missile = (100 - tan_phi) * 2.0 / 3.0
            if t_missile < 0: continue
            numerator = (300 * t_missile - 2200) * cos_phi
            denominator = v * np.cos(phi + beta)
            if abs(denominator) < 1e-9: continue
            t1_plus_t2 = numerator / denominator
            t3 = t_missile - t1_plus_t2
            if 0 <= t3 <= 20:
                valid_solutions.append((v, alpha, phi, t3))
            print(f"找到 {len(valid_solutions)} 个有效点")
        return valid_solutions

def convert_points(zero_points):

    solutions = []
    for v, alpha, phi, t3 in zero_points:
        t_missile = (100 - np.tan(phi)) * 2.0 / 3.0
        T = t_missile - t3 # T = t_drop + t_delay
        if T < 0: continue

        t2_ratio = random.uniform(0.1, 0.6)
        t2 = T * t2_ratio
        t1 = T - t2

        if t1 > 0 and t2 > 0:
            solutions.append({
                "v": v, "alpha_deg": np.rad2deg(alpha),
                "t_drop": t1, "t_delay": t2
            })
    print(f"转化了 {len(solutions)} 个")
    return solutions

def create_diverse_initial_population(solutions, num_individuals, consts_tuple):

    print(f"构建 {num_individuals} 个种群")
    if not solutions: return None

    grouped_solutions = {}
    for s in solutions:
        key = (round(s['v']), round(s['alpha_deg']))
        if key not in grouped_solutions:
            grouped_solutions[key] = []
        grouped_solutions[key].append(s)

    initial_population = []

    QUALITY_THRESHOLD = 4.0
    print(f"质量门槛为: {QUALITY_THRESHOLD} s")

    sorted_groups = sorted(grouped_solutions.items(), key=lambda item: len(item[1]),
                           reverse=True)

```

```

for (v, alpha_deg), group in sorted_groups:
    if len(group) < 3:
        continue

    for combo in combinations(group, 3):
        s1, s2, s3 = sorted(combo, key=lambda x: x['t_drop'])

        delta_t2 = s2['t_drop'] - s1['t_drop']
        delta_t3 = s3['t_drop'] - s2['t_drop']

        if delta_t2 >= 1.0 and delta_t3 >= 1.0:

            candidate_params = [
                v, alpha_deg,
                s1['t_drop'], s1['t_delay'],
                delta_t2, s2['t_delay'],
                delta_t3, s3['t_delay']
            ]

            shield_time, _ = calculate_time3(np.array(candidate_params), consts_tuple)

            if shield_time > QUALITY_THRESHOLD:
                initial_population.append(candidate_params)

            if len(initial_population) > num_individuals * 2:
                break
            if len(initial_population) > num_individuals * 2:
                break

            print(f"通过质量过滤, 成功构建 {len(initial_population)} 个高质量初始个体。")
            if not initial_population: return None

            if len(initial_population) > num_individuals:
                initial_population = [initial_population[i] for i in
                    random.sample(range(len(initial_population)), num_individuals)]

            elif len(initial_population) > 0:
                while len(initial_population) < num_individuals:
                    initial_population.append(initial_population[random.randint(0,
                        len(initial_population)-1)])

            return np.array(initial_population)

class Question3SolverDE:
    def __init__(self):
        self.config = Config()
        self.ml_vector = (self.config.fake_target - self.config.ml_init_pos) /
            np.linalg.norm(self.config.ml_init_pos - self.config.fake_target)
        self.missile_flight_time = np.linalg.norm(self.config.fake_target -
            self.config.ml_init_pos) / self.config.missile_v
        self.consts = (
            self.config.fyl_init_pos, self.config.smoke_sink_v, self.config.sim_step,
            self.config.ml_init_pos, self.ml_vector, self.config.missile_v,
            self.config.target_base_pos, self.config.cloud_r, self.config.g,
            self.missile_flight_time, self.config.target_r, self.config.target_h
        )

    def objective_func(self, params):

```

```

total_time, _ = calculate_time3(params, self.consts)
return -total_time

def solve(self, maxiter=100, popsize=15, mutation=(0.5, 1), recombination=0.7):
    bounds = [
        (70, 140),
        (0, 180),
        (0.1, 40),
        (0.1, 5),
        (1.0, 15),
        (0.1, 5),
        (1.0, 15),
        (0.1, 10)
    ]

    zero_points = find_zeropoints(v_samples=70, alpha_samples=180, phi_samples=200)
    single_solutions = convert_points(zero_points)
    init_population = create_diverse_initial_population(single_solutions,
        num_individuals=popsize, consts_tuple=self.consts)

    if init_population is None or len(init_population) < popsize:
        init_population = 'latinhypercube'
    else:
        for i in range(len(init_population)):
            for j in range(len(bounds)):
                init_population[i][j] = np.clip(init_population[i][j], bounds[j][0],
                    bounds[j][1])

    result = differential_evolution(
        self.objective_func,
        bounds,
        strategy='randlbin',
        maxiter=maxiter,
        popsize=popsize,
        mutation=mutation,
        recombination=recombination,
        disp=True,
        workers=-1,
        updating='deferred',
        polish=True,
        init=init_population,
    )

    total_shield_time, individual_times = calculate_time3(result.x, self.consts)
    return result.x, total_shield_time, individual_times

def print_results(self, params, total_shield_time, individual_times):
    v_fyl, alpha_deg, t_drop1, t_delay1, delta_t_drop2, t_delay2, delta_t_drop3,
        t_delay3 = params
    alpha_rad = math.radians(alpha_deg)
    fyl_vector = np.array([np.cos(alpha_rad), np.sin(alpha_rad), 0.0])
    v_at_drop = fyl_vector * v_fyl
    t_drop = np.array([t_drop1, t_drop1 + delta_t_drop2, t_drop1 + delta_t_drop2 +
        delta_t_drop3])
    t_delay = np.array([t_delay1, t_delay2, t_delay3])

```

```

print("\n" + "="*35 + " 优化结果 " + "="*35)
print(f"{'无人机运动方向':<20}: {alpha_deg:.3f} 度")
print(f"{'无人机运动速度 (m/s)':<20}: {v_fy1:.3f}")
print(f"{'总有效干扰时长 (s)':<20}: {total_shield_time:.3f}")
print("-" * 80)

for i in range(3):
    print(f" - 干扰弹 {i+1} 单独干扰时长: {individual_times[i]:.3f} s")
    print("-" * 80)

header = (f"{'编号':<5}{'投放点x (m)':>15}{'投放点y (m)':>15}{'投放点z (m)':>15}"
f"{'起爆点x (m)':>15}{'起爆点y (m)':>15}{'起爆点z (m)':>15}")
print(header)
print("-" * 105)

for i in range(3):
    drop_pos = self.config.fy1_init_pos + fy1_vector * v_fy1 * t_drop[i]
    detonation_pos = drop_pos + v_at_drop * t_delay[i] + 0.5 * np.array([0.0, 0.0,
        -self.config.g]) * t_delay[i]**2
    row_str =
        (f"{i+1:<5}{drop_pos[0]:>15.3f}{drop_pos[1]:>15.3f}{drop_pos[2]:>15.3f}"
f"{detonation_pos[0]:>15.3f}{detonation_pos[1]:>15.3f}{detonation_pos[2]:>15.3f}")
    print(row_str)

def main():
    solver = Question3SolverDE()

    print("="*25 + " 开始优化 " + "="*25)
    print(f"导弹预计飞行时间: {solver.missile_flight_time:.2f} s")

    best_params, total_shield_time, individual_times = solver.solve(
        maxiter=2000,
        popsize=100,
        mutation=(0.7, 1.5),
        recombination=0.8
    )

    print("\n" + "="*25 + " 优化完成 " + "="*25)
    if total_shield_time > 0 and best_params is not None:
        print(f"找到的最大遮蔽时间: {total_shield_time:.4f} s")
        print("\n详细参数:")
        param_names = ["v_fy1", "alpha_deg", "t_drop1", "t_delay1", "delta_t_drop2",
            "t_delay2", "delta_t_drop3", "t_delay3"]
        for name, val in zip(param_names, best_params):
            print(f" - {name}: {val:.4f}")

    solver.print_results(best_params, total_shield_time, individual_times)
    else:
        print("未找到有效策略")

if __name__ == '__main__':
    main()

```

附录 D 问题三验证代码

```

import numpy as np
import math
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
from numba import jit
import pandas as pd

@jit(nopython=True)
def check_intersect(cone_vtx, cone_axis, cone_angle, cyl_base, cyl_r, cyl_h):
    cyl_top = cyl_base + np.array([0.0, 0.0, cyl_h])
    key_points = np.empty((64, 3))
    for i in range(32):
        angle = 2 * np.pi * i / 32
        offset = np.array([cyl_r * np.cos(angle), cyl_r * np.sin(angle), 0.0])
        key_points[i * 2] = cyl_base + offset
        key_points[i * 2 + 1] = cyl_top + offset

    has_inside, has_outside = False, False
    for i in range(64):
        p = key_points[i]
        vec_to_p = p - cone_vtx
        dist = np.linalg.norm(vec_to_p)
        if dist < 1e-9: return True

    cos_val = np.dot(vec_to_p, cone_axis) / dist
    angle_to_axis = np.arccos(max(-1.0, min(cos_val, 1.0)))

    if angle_to_axis < cone_angle:
        has_inside = True
    else:
        has_outside = True

    if has_inside and has_outside:
        return True
    return False

@jit(nopython=True)
def is_shielded(m_pos, s_pos, s_r, target_base, target_r, target_h):
    vec_m_to_s = s_pos - m_pos
    dist_m_to_s = np.linalg.norm(vec_m_to_s)

    if dist_m_to_s <= s_r: return False

    cone_vtx = m_pos
    cone_axis = vec_m_to_s / dist_m_to_s
    cone_angle = np.arcsin(min(s_r / dist_m_to_s, 1.0))

    if check_intersect(cone_vtx, cone_axis, cone_angle, target_base, target_r,
        target_h):
        return False

    vec_vtx_to_target = target_base - cone_vtx
    dist_to_axis = np.linalg.norm(np.cross(vec_vtx_to_target, cone_axis))
    proj_dist = np.dot(vec_vtx_to_target, cone_axis)

    if proj_dist <= 0: return False

    cone_radius_at_target = proj_dist * np.tan(cone_angle)

```

```

return dist_to_axis < cone_radius_at_target

def plot_and_analyze_q3_results(best_params):
    # --- 配置和常量 ---
    missile_v = 300.0
    g = 9.8
    cloud_r = 10.0
    smoke_sink_v = np.array([0.0, 0.0, -3.0])
    sim_step = 0.0001 # 使用更精确的仿真步长

    fake_target = np.array([0.0, 0.0, 0.0])
    m1_init_pos = np.array([20000.0, 0.0, 2000.0])
    fyl_init_pos = np.array([17800.0, 0.0, 1800.0])

    target_base_pos = np.array([0.0, 200.0, 0.0])
    target_r = 7.0
    target_h = 10.0

    m1_vector = (fake_target - m1_init_pos) / np.linalg.norm(m1_init_pos -
        fake_target)
    missile_flight_time = np.linalg.norm(fake_target - m1_init_pos) / missile_v

    # --- 解包最优参数 ---
    v_fyl, alpha_deg, t_drop1, t_delay1, delta_t_drop2, t_delay2, delta_t_drop3,
        t_delay3 = best_params

    # --- 计算关键位置和时间 ---
    alpha_rad = math.radians(alpha_deg)
    fyl_vector = np.array([np.cos(alpha_rad), np.sin(alpha_rad), 0.0])
    v_at_drop = fyl_vector * v_fyl

    t_drop = np.array([t_drop1, t_drop1 + delta_t_drop2, t_drop1 + delta_t_drop2 +
        delta_t_drop3])
    t_delay = np.array([t_delay1, t_delay2, t_delay3])
    t_detonation = t_drop + t_delay

    drop_pos = np.zeros((3, 3))
    detonation_pos = np.zeros((3, 3))
    for i in range(3):
        drop_pos[i] = fyl_init_pos + fyl_vector * v_fyl * t_drop[i]
        detonation_pos[i] = drop_pos[i] + v_at_drop * t_delay[i] + 0.5 * np.array([0.0,
            0.0, -g]) * t_delay[i]**2

    # --- 仿真并记录数据 ---
    times = np.arange(0, missile_flight_time, sim_step)
    missile_path = np.array([m1_init_pos + m1_vector * missile_v * t for t in times])
    drone_path = np.array([fyl_init_pos + fyl_vector * v_fyl * t for t in times if t
        <= max(t_drop)])

    shielded_times = []
    shielding_intervals = []
    in_shield = False
    individual_shield_times = np.zeros(3)
    simulation_data = []

    for t in times:
        m_pos = m1_init_pos + m1_vector * missile_v * t

```

```

drone_pos = fyl_init_pos + fyl_vector * v_fyl * t
is_shielded_at_t = False

s_pos_all = np.full((3, 3), np.nan)

shielding_jammers_at_t = np.zeros(3, dtype=np.bool_)
for i in range(3):
    if t >= t_detonation[i] and t <= t_detonation[i] + 20.0:
        s_pos = detonation_pos[i] + smoke_sink_v * (t - t_detonation[i])
        s_pos_all[i, :] = s_pos
        if is_shielded(m_pos, s_pos, cloud_r, target_base_pos, target_r, target_h):
            shielding_jammers_at_t[i] = True
            is_shielded_at_t = True

simulation_data.append({
    'time': t,
    'drone_x': drone_pos[0], 'drone_y': drone_pos[1], 'drone_z': drone_pos[2],
    'smoke1_x': s_pos_all[0, 0], 'smoke1_y': s_pos_all[0, 1], 'smoke1_z':
        s_pos_all[0, 2],
    'smoke2_x': s_pos_all[1, 0], 'smoke2_y': s_pos_all[1, 1], 'smoke2_z':
        s_pos_all[1, 2],
    'smoke3_x': s_pos_all[2, 0], 'smoke3_y': s_pos_all[2, 1], 'smoke3_z':
        s_pos_all[2, 2],
    'missile_x': m_pos[0], 'missile_y': m_pos[1], 'missile_z': m_pos[2],
    'is_shielded': is_shielded_at_t
})

# 更新总干扰区间
if is_shielded_at_t and not in_shield:
    in_shield = True
    shielding_intervals.append([t])
elif not is_shielded_at_t and in_shield:
    in_shield = False
    shielding_intervals[-1].append(t)

# 累加单个干扰时长
for i in range(3):
    if shielding_jammers_at_t[i]:
        individual_shield_times[i] += sim_step

if in_shield:
    shielding_intervals[-1].append(missile_flight_time)

df = pd.DataFrame(simulation_data)
df.to_excel("q3_simulation_data.xlsx", index=False)
print("\nSimulation data has been exported to q3_simulation_data.xlsx")

# --- 打印精确干扰时间 ---
print("\n" + "="*30 + " 精确干扰时间分析 " + "="*30)
if not shielding_intervals:
    print("未检测到有效干扰。")
else:
    total_duration = 0
    for i, interval in enumerate(shielding_intervals):
        duration = interval[1] - interval[0]
        total_duration += duration
    print(f" - 干扰区间 {i+1}: 从 {interval[0]:.4f} s 到 {interval[1]:.4f} s (时长: {duration:.4f} s)")

```

```

print(f"\n总干扰时长 (精确计算): {total_duration:.4f} s")
print("-" * 82)
print("各干扰弹单独贡献时长:")
for i in range(3):
    print(f" - 干扰弹 {i+1}: {individual_shield_times[i]:.4f} s")

print("-" * 82)

# --- 计算高度差 ---
print("\n" + "="*28 + " 爆炸点与导弹-目标平面高度差分析 " + "="*28)
for i in range(3):
    # 爆炸瞬间的导弹位置
    t_det = t_detonation[i]
    missile_pos_at_det = m1_init_pos + m1_vector * missile_v * t_det

    # 定义平面上的三个点
    p1 = missile_pos_at_det # 导弹位置
    p2 = target_base_pos # 真目标位置
    p3 = m1_init_pos # 导弹初始位置

    # 计算平面法向量
    vec1 = p2 - p1
    vec2 = p3 - p1
    normal_vector = np.cross(vec1, vec2)
    normal_vector = normal_vector / np.linalg.norm(normal_vector)

    # 计算爆炸点到平面的距离 (高度差)
    point_on_plane = p1
    vec_to_detonation = detonation_pos[i] - point_on_plane
    height_difference = np.abs(np.dot(vec_to_detonation, normal_vector))

    print(f" - 干扰弹 {i+1} (t={t_det:.2f}s):")
    print(f" - 爆炸点与'导弹-真目标-初始点'平面的高度差: {height_difference:.3f} m")
    print("-" * 82)

# --- 3D 可视化 ---
fig = plt.figure(figsize=(12, 9))
ax = fig.add_subplot(111, projection='3d')

# 绘制轨迹
ax.plot(missile_path[:, 0], missile_path[:, 1], missile_path[:, 2],
        label='Missile Path', color='red', linestyle='--')
ax.plot(drone_path[:, 0], drone_path[:, 1], drone_path[:, 2], label='Drone Path', color='blue')

# 绘制关键点
ax.scatter(m1_init_pos[0], m1_init_pos[1], m1_init_pos[2], color='red', s=100,
           marker='^', label='Missile Start')
ax.scatter(fy1_init_pos[0], fy1_init_pos[1], fy1_init_pos[2], color='blue',
           s=100, marker='^', label='Drone Start')
ax.scatter(target_base_pos[0], target_base_pos[1], target_base_pos[2],
           color='green', s=150, marker='s', label='Target')

# 绘制投放点和爆炸点
ax.scatter(drop_pos[:, 0], drop_pos[:, 1], drop_pos[:, 2], c='cyan', s=50,
           marker='o', label='Drop Points')
ax.scatter(detonation_pos[:, 0], detonation_pos[:, 1], detonation_pos[:, 2],

```



```

c='magenta', s=80, marker='*', label='Detonation Points')

# 绘制目标圆柱
u = np.linspace(0, 2 * np.pi, 50)
h = np.linspace(0, target_h, 20)
x = np.outer(target_r * np.cos(u), np.ones_like(h)) + target_base_pos[0]
y = np.outer(target_r * np.sin(u), np.ones_like(h)) + target_base_pos[1]
z = np.outer(np.ones_like(u), h) + target_base_pos[2]
ax.plot_surface(x, y, z, color='green', alpha=0.3)

ax.set_xlabel('X (m)')
ax.set_ylabel('Y (m)')
ax.set_zlabel('Z (m)')
ax.set_title('3D Visualization of Q3 Optimal Strategy')
ax.legend()
ax.grid(True)

# 设置合适的坐标轴范围以获得更好的视图
all_points = np.vstack([missile_path, drone_path, np.array([target_base_pos])])
min_coords = all_points.min(axis=0)
max_coords = all_points.max(axis=0)
ax.set_xlim(min_coords[0], max_coords[0])
ax.set_ylim(min_coords[1], max_coords[1])
ax.set_zlim(0, max_coords[2])

plt.show()

if __name__ == '__main__':
# 从 q3_de.py 运行得到的最佳参数
# ===== 优化完成 =====
# 找到的最优策略提供的最大遮蔽时间: 7.4000 s
#
# 详细参数:
# - v_fy1: 138.5400
# - alpha_deg: 179.6306
# - t_drop1: 0.2342
# - t_delay1: 3.8390
# - delta_t_drop2: 2.9163
# - t_delay2: 4.9846
# - delta_t_drop3: 2.0803
# - t_delay3: 5.8581
optimal_params = [
138.5400, 179.6306, 0.2342, 3.8390,
2.9163, 4.9846, 2.0803, 5.8581
]

plot_and_analyze_q3_results(optimal_params)

```

附录 E 问题四求解代码

```

import numpy as np
import math
import pandas as pd
from scipy.optimize import differential_evolution
from numba import jit

```

```

@jit(nopython=True)
def check_intersect(cone_vtx, cone_axis, cone_angle, cyl_base, cyl_r, cyl_h):
    cyl_top = cyl_base + np.array([0.0, 0.0, cyl_h])
    key_points = np.empty((64, 3))
    for i in range(32):
        angle = 2 * np.pi * i / 32
        offset = np.array([cyl_r * np.cos(angle), cyl_r * np.sin(angle), 0.0])
        key_points[i * 2] = cyl_base + offset
        key_points[i * 2 + 1] = cyl_top + offset
        has_inside, has_outside = False, False
    for i in range(64):
        p = key_points[i]
        vec_to_p = p - cone_vtx
        dist = np.linalg.norm(vec_to_p)
        if dist < 1e-9: return True
        cos_val = np.dot(vec_to_p, cone_axis) / dist
        angle_to_axis = np.arccos(max(-1.0, min(cos_val, 1.0)))

        if angle_to_axis < cone_angle:
            has_inside = True
        else:
            has_outside = True

    if has_inside and has_outside:
        return True
    return False

@jit(nopython=True)
def is_shielded(m_pos, s_pos, s_r, target_base, target_r, target_h):
    vec_m_to_s = s_pos - m_pos
    dist_m_to_s = np.linalg.norm(vec_m_to_s)

    if dist_m_to_s <= s_r: return False

    cone_vtx = m_pos
    cone_axis = vec_m_to_s / dist_m_to_s
    cone_angle = np.arcsin(min(s_r / dist_m_to_s, 1.0))

    if check_intersect(cone_vtx, cone_axis, cone_angle, target_base, target_r,
        target_h):
        return False

    vec_vtx_to_target = target_base - cone_vtx
    dist_to_axis = np.linalg.norm(np.cross(vec_vtx_to_target, cone_axis))
    proj_dist = np.dot(vec_vtx_to_target, cone_axis)

    if proj_dist <= 0: return False

    cone_radius_at_target = proj_dist * np.tan(cone_angle)
    return dist_to_axis < cone_radius_at_target

@jit(nopython=True)
def calculate_time_q4(params, consts):
    v_fyl, alpha1, t_drop1, t_delay1, \

```

```

v_fy2, alpha2, t_drop2, t_delay2, \
v_fy3, alpha3, t_drop3, t_delay3 = params

fy_init_pos, smoke_sink_v, sim_step, m1_init_pos, m1_vector, m_v, target_base,
    s_r, g, m_flight_time, target_r, target_h = consts

fy_params = np.array([
[v_fy1, alpha1, t_drop1, t_delay1],
[v_fy2, alpha2, t_drop2, t_delay2],
[v_fy3, alpha3, t_drop3, t_delay3]
])

detonation_pos = np.zeros((3, 3))
t_detonation = np.zeros(3)

for i in range(3):
v_fy, alpha_deg, t_drop, t_delay = fy_params[i]
alpha_rad = math.radians(alpha_deg)
fy_vector = np.array([np.cos(alpha_rad), np.sin(alpha_rad), 0.0])
v_at_drop = fy_vector * v_fy

drop_pos = fy_init_pos[i] + fy_vector * v_fy * t_drop
detonation_pos[i] = drop_pos + v_at_drop * t_delay + 0.5 * np.array([0.0, 0.0,
    -g]) * t_delay**2
t_detonation[i] = t_drop + t_delay

total_shield_time = 0.0
for t in np.arange(0, m_flight_time, sim_step):
m_pos = m1_init_pos + m1_vector * m_v * t
is_shielded_at_t = False
for i in range(3):
if t >= t_detonation[i] and t <= t_detonation[i] + 20.0:
s_pos = detonation_pos[i] + smoke_sink_v * (t - t_detonation[i])
if is_shielded(m_pos, s_pos, s_r, target_base, target_r, target_h):
is_shielded_at_t = True
break

if is_shielded_at_t:
total_shield_time += sim_step

return total_shield_time

class Config:
def __init__(self):
self.missile_v = 300.0
self.g = 9.8
self.cloud_r = 10.0
self.smoke_sink_v = np.array([0.0, 0.0, -3.0])
self.sim_step = 0.1

self.fake_target = np.array([0.0, 0.0, 0.0])
self.m1_init_pos = np.array([20000.0, 0.0, 2000.0])

self.fy_init_pos = np.array([
[17800.0, 0.0, 1800.0], # FY1
[12000.0, 1400.0, 1400.0], # FY2
[6000.0, -3000.0, 700.0] # FY3
])

```

```

self.target_base_pos = np.array([0.0, 200.0, 0.0])
self.target_r = 7.0
self.target_h = 10.0

class Question4Solver:
def __init__(self):
self.config = Config()
self.ml_vector = (self.config.fake_target - self.config.ml_init_pos) /
    np.linalg.norm(self.config.ml_init_pos - self.config.fake_target)
self.missile_flight_time = np.linalg.norm(self.config.fake_target -
    self.config.ml_init_pos) / self.config.missile_v
self.consts = (
self.config.fy_init_pos, self.config.smoke_sink_v, self.config.sim_step,
self.config.ml_init_pos, self.ml_vector, self.config.missile_v,
self.config.target_base_pos, self.config.cloud_r, self.config.g,
self.missile_flight_time, self.config.target_r, self.config.target_h
)
def objective_func(self, params):
return -calculate_time_q4(params, self.consts)

def solve(self):
bounds = [
(70, 140), (179, 180), (0.1, 40), (0.1, 10), # FY1
(70, 140), (180, 360), (0.1, 40), (0.1, 10), # FY2
(70, 140), (0, 180), (0.1, 40), (0.1, 10) # FY3
]

result = differential_evolution(
self.objective_func,
bounds,
strategy='best1bin',
maxiter=600,
popsize=100,
mutation=(0.5, 1),
recombination=0.7,
updating='deferred',
workers=-1,
disp=True
)

if result.fun < 0:
return result.x, -result.fun
else:
print("未能找有效解")
return None, 0

def print_results_table(self, params, shield_time):

fy_params = np.array(params).reshape(3, 4)

data = []
for i in range(3):
v_fy, alpha_deg, t_drop, t_delay = fy_params[i]
alpha_rad = math.radians(alpha_deg)
fy_vector = np.array([np.cos(alpha_rad), np.sin(alpha_rad), 0.0])
v_at_drop = fy_vector * v_fy

```

```

drop_pos = self.config.fy_init_pos[i] + fy_vector * v_fy * t_drop
detonation_pos = drop_pos + v_at_drop * t_delay + 0.5 * np.array([0.0, 0.0,
    -self.config.g]) * t_delay**2

data.append({
    "编号": f"FY{i+1}",
    "方向": f"{alpha_deg:.4f}",
    "速度 (m/s)": f"{v_fy:.4f}",
    "投放点x": f"{drop_pos[0]:.2f}",
    "投放点y": f"{drop_pos[1]:.2f}",
    "投放点z": f"{drop_pos[2]:.2f}",
    "起爆点x": f"{detonation_pos[0]:.2f}",
    "起爆点y": f"{detonation_pos[1]:.2f}",
    "起爆点z": f"{detonation_pos[2]:.2f}",
    "总干扰时长/s": f"{shield_time:.4f}" if i == 0 else ""
})

df = pd.DataFrame(data)
print("\n" + " 优化结果详情 ")
print(df.to_string())

def main():
    solver = Question4Solver()

    print(f"导弹飞行时间: {solver.missile_flight_time:.2f} s")

    best_params, max_shield_time = solver.solve()

    print("\n 优化完成 ")
    if max_shield_time > 0 and best_params is not None:
        print(f"最大遮蔽时间: {max_shield_time:.4f} s")
        param_names = [
            "v_fy1", "alpha1", "t_drop1", "t_delay1",
            "v_fy2", "alpha2", "t_drop2", "t_delay2",
            "v_fy3", "alpha3", "t_drop3", "t_delay3"
        ]
        for name, val in zip(param_names, best_params):
            print(f"{name}: {val:.4f}")

    solver.print_results_table(best_params, max_shield_time)
    else:
        print("未找到有效解")

if __name__ == '__main__':
    main()

```

附录 F 问题四验证代码

```

import numpy as np
import math
import pandas as pd
from numba import jit
import matplotlib.pyplot as plt
from matplotlib.patches import Circle

@jit(nopython=True)

```

```

def check_intersect(cone_vtx, cone_axis, cone_angle, cyl_base, cyl_r, cyl_h):
    cyl_top = cyl_base + np.array([0.0, 0.0, cyl_h])
    key_points = np.empty((64, 3))
    for i in range(32):
        angle = 2 * np.pi * i / 32
        offset = np.array([cyl_r * np.cos(angle), cyl_r * np.sin(angle), 0.0])
        key_points[i * 2] = cyl_base + offset
        key_points[i * 2 + 1] = cyl_top + offset

    has_inside, has_outside = False, False
    for i in range(64):
        p = key_points[i]
        vec_to_p = p - cone_vtx
        dist = np.linalg.norm(vec_to_p)
        if dist < 1e-9: return True

    cos_val = np.dot(vec_to_p, cone_axis) / dist
    angle_to_axis = np.arccos(max(-1.0, min(cos_val, 1.0)))

    if angle_to_axis < cone_angle:
        has_inside = True
    else:
        has_outside = True

    if has_inside and has_outside:
        return True
    return False

@jit(nopython=True)
def is_shielded(m_pos, s_pos, s_r, target_base, target_r, target_h):

    vec = s_pos - m_pos
    dist = np.linalg.norm(vec)

    if dist <= s_r: return False

    cone_vtx = m_pos
    cone_axis = vec / dist
    cone_angle = np.arcsin(min(s_r / dist, 1.0))

    if check_intersect(cone_vtx, cone_axis, cone_angle, target_base, target_r,
        target_h):
        return False

    vec_target = target_base - cone_vtx
    dist_to_axis = np.linalg.norm(np.cross(vec_target, cone_axis))
    proj_dist = np.dot(vec_target, cone_axis)

    if proj_dist <= 0: return False

    radius = proj_dist * np.tan(cone_angle)
    return dist_to_axis < radius

class Config:
    def __init__(self):
        self.missile_v = 300.0
        self.g = 9.8

```

```

self.cloud_r = 10.0
self.smoke_sink_v = np.array([0.0, 0.0, -3.0])
self.sim_step = 0.0001 # 使用高精度仿真步长

self.fake_target = np.array([0.0, 0.0, 0.0])
self.m1_init_pos = np.array([20000.0, 0.0, 2000.0])

self.fy_init_pos = {
    'FY1': np.array([17800.0, 0.0, 1800.0]),
    'FY2': np.array([12000.0, 1400.0, 1400.0]),
    'FY3': np.array([6000.0, -3000.0, 700.0])
}

self.target_base_pos = np.array([0.0, 200.0, 0.0])
self.target_r = 7.0
self.target_h = 10.0

def analyze_strategy(params, config):

    fy_params_arr = np.array(params).reshape(3, 4)
    fy_ids = ['FY1', 'FY2', 'FY3']

    m1_vector = (config.fake_target - config.m1_init_pos) /
        np.linalg.norm(config.m1_init_pos - config.fake_target)
    missile_flight_time = np.linalg.norm(config.fake_target - config.m1_init_pos) /
        config.missile_v
    print(f"导弹飞行时间: {missile_flight_time:.4f} s")
    print(f"仿真步长: {config.sim_step} s")

    detonation_pos_list = []
    t_detonation_list = []

    results_data = []

    for i in range(3):
        v_fy, alpha_deg, t_drop, t_delay = fy_params_arr[i]
        fy_id = fy_ids[i]
        fy_init_pos = config.fy_init_pos[fy_id]

        alpha_rad = math.radians(alpha_deg)
        fy_vector = np.array([np.cos(alpha_rad), np.sin(alpha_rad), 0.0])
        v_at_drop = fy_vector * v_fy

        drop_pos = fy_init_pos + fy_vector * v_fy * t_drop
        detonation_pos = drop_pos + v_at_drop * t_delay + 0.5 * np.array([0.0, 0.0,
            -config.g]) * t_delay**2
        t_detonation = t_drop + t_delay

        detonation_pos_list.append(detonation_pos)
        t_detonation_list.append(t_detonation)

    results_data.append({
        "编号": fy_id,
        "速度": f"{v_fy:.4f}",
        "方向": f"{alpha_deg:.4f}",
        "t1(s)": f"{t_drop:.4f}",
        "t2(s)": f"{t_delay:.4f}",
        "t3(s)": f"{t_detonation:.4f}",
    })

```

```

        "投放点(x,y,z)": f"({drop_pos[0]:.2f}, {drop_pos[1]:.2f}, {drop_pos[2]:.2f})",
        "起爆点(x,y,z)": f"({detonation_pos[0]:.2f}, {detonation_pos[1]:.2f},
            {detonation_pos[2]:.2f})"
    })

df = pd.DataFrame(results_data)
print(df.to_string())

total_shield_time = 0.0
individual_shield_times = {fy_id: 0.0 for fy_id in fy_ids}
simulation_data_q4 = []

fy_vectors = []
for i in range(3):
    v_fy, alpha_deg, _, _ = fy_params_arr[i]
    alpha_rad = math.radians(alpha_deg)
    fy_vectors.append(np.array([np.cos(alpha_rad), np.sin(alpha_rad), 0.0]) * v_fy)

for t in np.arange(0, missile_flight_time, config.sim_step):
    m_pos = config.m1_init_pos + m1_vector * config.missile_v * t

    drone_positions = {
        'FY1': config.fy_init_pos['FY1'] + fy_vectors[0] * t,
        'FY2': config.fy_init_pos['FY2'] + fy_vectors[1] * t,
        'FY3': config.fy_init_pos['FY3'] + fy_vectors[2] * t
    }

    is_shielded_at_t = False
    smoke_positions = {}
    for i in range(3):
        fy_id = fy_ids[i]
        if t >= t_detonation_list[i] and t <= t_detonation_list[i] + 20.0:
            s_pos = detonation_pos_list[i] + config.smoke_sink_v * (t - t_detonation_list[i])
            smoke_positions[fy_id] = s_pos
            if is_shielded(m_pos, s_pos, config.cloud_r, config.target_base_pos,
                config.target_r, config.target_h):
                is_shielded_at_t = True
            individual_shield_times[fy_id] += config.sim_step
        else:
            smoke_positions[fy_id] = np.array([np.nan, np.nan, np.nan])

    if is_shielded_at_t:
        total_shield_time += config.sim_step

    row_data = {'time': t, 'missile_x': m_pos[0], 'missile_y': m_pos[1],
        'missile_z': m_pos[2]}
    for i, fy_id in enumerate(fy_ids):
        row_data[f'drone{i+1}_x'] = drone_positions[fy_id][0]
        row_data[f'drone{i+1}_y'] = drone_positions[fy_id][1]
        row_data[f'drone{i+1}_z'] = drone_positions[fy_id][2]
        row_data[f'smoke{i+1}_x'] = smoke_positions[fy_id][0]
        row_data[f'smoke{i+1}_y'] = smoke_positions[fy_id][1]
        row_data[f'smoke{i+1}_z'] = smoke_positions[fy_id][2]
        row_data['is_shielded'] = is_shielded_at_t
    simulation_data_q4.append(row_data)

# df_sim = pd.DataFrame(simulation_data_q4)

```



```

# df_sim.to_excel("q4_simulation_data.xlsx", index=False)
# print("\nQ4 simulation data has been exported to q4_simulation_data.xlsx")

print("\n分析结果 ")
print(f"总遮蔽时间: {total_shield_time:.4f} s")
for fy_id, shield_time in individual_shield_times.items():
    print(f"{fy_id} 的有效干扰时间: {shield_time:.4f} s")

plot_q4_top_view(fy_params_arr, detonation_pos_list, config)

def plot_q4_top_view(params_list, detonation_pos_list, config):
    plt.figure(figsize=(12, 10))
    ax = plt.gca()

    m1_vector = (config.fake_target - config.m1_init_pos) /
        np.linalg.norm(config.m1_init_pos - config.fake_target)
    m_path = np.array([config.m1_init_pos, config.fake_target])
    ax.plot(m_path[:, 0], m_path[:, 1], 'r--', label="Missile M1 Path")
    ax.scatter(config.m1_init_pos[0], config.m1_init_pos[1], c='red', s=100,
        label="M1 Start", zorder=5)

    ax.scatter(config.fake_target[0], config.fake_target[1], c='black', s=100,
        label="Fake Target", zorder=5)
    ax.scatter(config.target_base_pos[0], config.target_base_pos[1], c='green',
        s=100, label="True Target", zorder=5)
    ax.add_patch(Circle((config.target_base_pos[0], config.target_base_pos[1]),
        config.target_r, color='green', fill=False, ls='--'))

    colors = ['blue', 'purple', 'orange']
    fy_ids = ['FY1', 'FY2', 'FY3']
    for i in range(len(params_list)):
        v_fy, alpha_deg, t_drop, _ = params_list[i]
        fy_init_pos = config.fy_init_pos[fy_ids[i]]
        alpha_rad = math.radians(alpha_deg)
        fy_vector = np.array([np.cos(alpha_rad), np.sin(alpha_rad)])

        drop_pos_2d = fy_init_pos[:2] + fy_vector * v_fy * t_drop
        uav_path = np.array([fy_init_pos[:2], drop_pos_2d])
        ax.plot(uav_path[:, 0], uav_path[:, 1], '--', color=colors[i],
            label=f"{fy_ids[i]} Path")
        ax.scatter(fy_init_pos[0], fy_init_pos[1], c=colors[i], s=100,
            label=f"{fy_ids[i]} Start", zorder=5)

        det_pos = detonation_pos_list[i]
        ax.scatter(det_pos[0], det_pos[1], c=colors[i], marker='x', s=150,
            label=f"{fy_ids[i]} Detonation", zorder=5)
        ax.add_patch(Circle((det_pos[0], det_pos[1]), config.cloud_r, color=colors[i],
            alpha=0.3))

    ax.set_xlabel("X (m)")
    ax.set_ylabel("Y (m)")
    ax.set_title("Question 4")
    ax.legend()

```

```

ax.grid(True)
ax.set_aspect('equal', adjustable='box')
plt.show()

def main():

    # OPTIMAL_PARAMS = [
    #     86.3464, 7.9778, 0.2116, 0.4483,
    #     97.0671, 295.9129, 10.7029, 5.0302, # alpha, t_drop, t_delay
    #     128.8553, 84.7080, 20.5872, 3.5507
    # ]
    OPTIMAL_PARAMS = [
    95.4923, 179.0354, 0.6701, 3.2220,
    121.3295, 293.9743, 7.0775, 5.1898, # alpha, t_drop, t_delay
    120.2815, 76.1727, 24.6815, 1.6813
    ]

    config = Config()
    analyze_strategy(OPTIMAL_PARAMS, config)

if __name__ == '__main__':
    main()

```

附录 G 问题五求解代码

```

import numpy as np
import math
from scipy.optimize import differential_evolution
from numba import jit
import warnings

warnings.filterwarnings("ignore", category=RuntimeWarning)

SEARCH_CONFIG = {

    'OPTIMIZE_ALL_UAVS': True,

    'SINGLE_UAV_ID': 'FY3'
}

# 'FY1': [
#     (130, 140), # v_fy
#     (179, 180), # alpha_deg
#     (0.1, 5), # t_drop1
#     (0.1, 5), # t_delay1
#     (1.0, 5), # delta_t_drop2 (必须 >= 1s)
#     (0.1, 5), # t_delay2
#     (1.0, 5), # delta_t_drop3 (必须 >= 1s)
#     (0.1, 10) # t_delay3
# ],
# 'FY2': [
#     (90, 100), (280, 290), (0.1, 5), (0.1, 5),
#     (1.0, 10), (0.1, 5), (1.0, 5), (0.1, 10)
# ],

```

```

#   'FY3': [
#       (130, 140), (70, 80), (0.1, 25), (0.1, 15),
#       (1.0, 15), (0.1, 15), (1.0, 10), (0.1, 15)
#   ],
#   'FY4': [
#       (129, 130), (279, 280), (3, 4), (11, 12),
#       (3, 4), (10, 11), (16, 19), (7, 10)
#   ],
#   'FY5': [
#       (139, 140), (120, 121), (12, 15), (0.1, 5),
#       (5.0, 7), (1, 2), (1.0, 2), (0.1, 2)
#   ],

# 参数: [v, alpha, t_drop1, t_delay1, delta_t2, t_delay2, delta_t3, t_delay3]
UAV_SEARCH_BOUNDS = {
    'FY1': [
        (70, 140),
        (0, 360),
        (0.1, 20),
        (0.1, 20),
        (1.0, 20),
        (0.1, 20),
        (1.0, 20),
        (0.1, 20)
    ],
    'FY2': [
        (70, 140),
        (0, 360),
        (0.1, 20),
        (0.1, 20),
        (1.0, 20),
        (0.1, 20),
        (1.0, 20),
        (0.1, 20)
    ],
    'FY3': [
        (70, 140),
        (0, 360),
        (0.1, 20),
        (0.1, 20),
        (1.0, 20),
        (0.1, 20),
        (1.0, 20),
        (0.1, 20)
    ],
    'FY4': [
        (70, 140),
        (0, 360),
        (0.1, 20),
        (0.1, 20),
        (1.0, 20),
        (0.1, 20),
        (1.0, 20),
        (0.1, 20)
    ],
    'FY5': [
        (70, 140),
        (0, 360),

```

```

(0.1, 20),
(0.1, 20),
(1.0, 20),
(0.1, 20),
(1.0, 20),
(0.1, 20)
],
# 'FY1': [
#     (130, 140), # v
#     (179, 180), # alpha
#     (0.1, 5), # t_d1
#     (0.1, 5), # t_delay1
#     (1.0, 5),
#     (0.1, 5),
#     (1.0, 5),
#     (0.1, 10) # t_delay3
# ],
# 'FY2': [
#     (90, 100), (280, 290), (0.1, 5), (0.1, 5),
#     (1.0, 10), (0.1, 5), (1.0, 5), (0.1, 10)
# ],
# 'FY3': [
#     (130, 140), (70, 80), (0.1, 25), (0.1, 15),
#     (1.0, 15), (0.1, 15), (1.0, 10), (0.1, 15)
# ],
# 'FY4': [
#     (129, 130), (279, 280), (3, 4), (11, 12),
#     (3, 4), (10, 11), (16, 19), (7, 10)
# ],
# 'FY5': [
#     (139, 140), (120, 121), (12, 15), (0.1, 5),
#     (5.0, 7), (1, 2), (1.0, 2), (0.1, 2)
# ],
}

@jit(nopython=True)
def is_shielded(m_pos, s_pos, s_r, target_pos):

    line_vec = target_pos - m_pos
    line_len_sq = np.dot(line_vec, line_vec)
    if line_len_sq < 1e-9:
        return False
    p_vec = s_pos - m_pos
    t = np.dot(p_vec, line_vec) / line_len_sq
    if t < 0.0 or t > 1.0:
        return False
    dist_sq = np.dot(p_vec, p_vec) - t * t * line_len_sq
    return dist_sq < s_r * s_r

class Config:
    def __init__(self):
        self.missile_v = 300.0
        self.g = 9.8
        self.cloud_r = 10.0
        self.smoke_sink_v = np.array([0.0, 0.0, -3.0])
        self.sim_step = 0.2

```

```

self.fake_target = np.array([0.0, 0.0, 0.0])
self.target_base_pos = np.array([0.0, 200.0, 0.0])
self.target_r = 7.0
self.target_h = 10.0

self.m_init_pos = {
    'M1': np.array([20000.0, 0.0, 2000.0]),
    'M2': np.array([19000.0, 600.0, 2100.0]),
    'M3': np.array([18000.0, -600.0, 1900.0])
}

self.fy_init_pos = {
    'FY1': np.array([17800.0, 0.0, 1800.0]),
    'FY2': np.array([12000.0, 1400.0, 1400.0]),
    'FY3': np.array([6000.0, -3000.0, 700.0]),
    'FY4': np.array([11000.0, 2000.0, 1800.0]),
    'FY5': np.array([13000.0, -2000.0, 1300.0])
}

self.m_vectors = {k: (self.fake_target - v) / np.linalg.norm(v -
    self.fake_target) for k, v in self.m_init_pos.items()}
self.m_flight_times = {k: np.linalg.norm(v - self.fake_target) / self.missile_v
    for k, v in self.m_init_pos.items()}

class Q5:
def __init__(self):
self.config = Config()
self.uav_ids = list(self.config.fy_init_pos.keys())
self.missile_ids = list(self.config.m_init_pos.keys())

def calculate_totaltime(self, params, target):
detonation_infos = []

param_idx = 0
for fy_id in self.uav_ids:
if fy_id in target:
v_fy, alpha_deg, t_drop1, t_delay1, delta_t2, t_delay2, delta_t3, t_delay3 =
    params[param_idx : param_idx + 8]
param_idx += 8

fy_init_pos = self.config.fy_init_pos[fy_id]
alpha_rad = math.radians(alpha_deg)
fy_vector = np.array([np.cos(alpha_rad), np.sin(alpha_rad), 0.0])
v_at_drop = fy_vector * v_fy

t_drops = np.array([t_drop1, t_drop1 + delta_t2, t_drop1 + delta_t2 + delta_t3])
t_delays = np.array([t_delay1, t_delay2, t_delay3])

for i in range(3):
t_drop = t_drops[i]
t_delay = t_delays[i]
drop_pos = fy_init_pos + fy_vector * v_fy * t_drop
detonation_pos = drop_pos + v_at_drop * t_delay + 0.5 * np.array([0.0, 0.0,
    -self.config.g]) * t_delay**2
t_detonation = t_drop + t_delay
detonation_infos.append({'pos': detonation_pos, 't_start': t_detonation})

```

```

if not detonation_infos:
    return 0

total_score = 0

# 为了简化模型，我们将真目标视为一个点（其几何中心）
target_pos = self.config.target_base_pos + np.array([0.0, 0.0,
    self.config.target_h / 2.0])

max_flight_time = 0
for m_id in self.missile_ids:
    if self.config.m_flight_times[m_id] > max_flight_time:
        max_flight_time = self.config.m_flight_times[m_id]

for t in np.arange(0, max_flight_time, self.config.sim_step):
    active_smokes_pos = []
    for smoke in detonation_infos:
        if t >= smoke['t_start'] and t <= smoke['t_start'] + 20.0:
            s_pos = smoke['pos'] + self.config.smoke_sink_v * (t - smoke['t_start'])
            active_smokes_pos.append(s_pos)

    if not active_smokes_pos:
        continue

    for m_id in self.missile_ids:
        if t > self.config.m_flight_times[m_id]:
            continue

    m_pos = self.config.m_init_pos[m_id] + self.config.m_vectors[m_id] *
        self.config.missile_v * t

    smoke_num = 0
    for s_pos in active_smokes_pos:
        if is_shielded(m_pos, s_pos, self.config.cloud_r, target_pos):
            smoke_num += 1

    total_score += smoke_num

return total_score * self.config.sim_step

def _objective_function(self, params, uav_ids_to_optimize):
    return -self.calculate_totaltime(params, uav_ids_to_optimize)

def run_optimization(self):
    if SEARCH_CONFIG['OPTIMIZE_ALL_UAVS']:
        print("--- 开始全局优化：5架无人机共同执行任务（40个参数） ---")
        uav_ids_to_optimize = self.uav_ids
        bounds = []
        for fy_id in uav_ids_to_optimize:
            bounds.extend(UAV_SEARCH_BOUNDS[fy_id])
        else:
            uav_id = SEARCH_CONFIG['SINGLE_UAV_ID']
            print(f"--- 开始单机优化：仅无人机 {uav_id}（8个参数） ---")
            uav_ids_to_optimize = [uav_id]
            bounds = UAV_SEARCH_BOUNDS[uav_id]

    result = differential_evolution(
        self._objective_function,

```

```

bounds,
args=(uav_ids_to_optimize,),
maxiter=1200, popsize=100,
workers=-1, disp=True, updating='deferred'
)

if result.fun < 0:
max_shield_time = -result.fun
print(f"\n>>> 优化成功! 最大总遮蔽分数: {max_shield_time:.2f}")
self.print_results(result.x, uav_ids_to_optimize)
else:
print("\n>>> 优化失败或未找到有效解。")

def print_results(self, params, uav_ids_to_optimize):
print("\n"+" 优化结果 ")
param_idx = 0
for fy_id in uav_ids_to_optimize:
print(f"\n--- 无人机: {fy_id} ---")
v_fy, alpha_deg, t_drop1, t_delay1, delta_t2, t_delay2, delta_t3, t_delay3 =
    params[param_idx : param_idx + 8]
param_idx += 8

print(f"速度: {v_fy:.2f} m/s\n 方向: {alpha_deg:.2f} degree")
print(f"烟1:")
print(f"t1: {t_drop1:.2f} s")
print(f"t2: {t_delay1:.2f} s")
print(f"烟2:")
print(f"t1: {delta_t2:.2f} s (t: {t_drop1 + delta_t2:.2f} s)")
print(f"t2: {t_delay2:.2f} s")
print(f"烟3:")
print(f"t1: {delta_t3:.2f} s (t: {t_drop1 + delta_t2 + delta_t3:.2f} s)")
print(f"t2: {t_delay3:.2f} s")

def main():
solver = Q5()
solver.run_optimization()

if __name__ == '__main__':
main()

```

附录 H 问题五验证代码

```

import numpy as np
import math
import pandas as pd
from numba import jit, prange

OPTIMAL_PARAMS = [
# FY1: [v, alpha, t1, t_delay1,
135.71, 179.43, 0.32, 3.78, 1.03, 4.19, 1.09, 4.79,
# FY2
93.03, 289.62, 0.40, 3.54, 8.47, 2.26, 1.47, 5.34,
# FY3
137.22, 75.19, 11.73, 5.39, 10.46, 1.08, 3.36, 1.72,
# FY4

```

```

129.71, 279.82, 3.79, 11.53, 3.78, 10.99, 16.90, 7.55,
# FY5
139.82, 120.05, 12.13, 4.59, 5.92, 1.61, 1.07, 0.54,
]

class Config:

    def __init__(self):
        self.G = 9.8
        self.MISSILE_V = 300.0
        self.SMOKE_SINK_V = np.array([0.0, 0.0, -3.0])
        self.CLOUD_R = 10.0
        self.SHIELD_DURATION = 20.0
        self.TARGET_R = 7.0
        self.TARGET_H = 10.0
        self.FAKE_TARGET_POS = np.array([0.0, 0.0, 0.0])
        self.TARGET_BASE_POS = np.array([0.0, 200.0, 0.0])

        self.target_feature_points = self.generate_points()

        self.M_INIT_POS = {
            'M1': np.array([20000.0, 0.0, 2000.0]),
            'M2': np.array([19000.0, 600.0, 2100.0]),
            'M3': np.array([18000.0, -600.0, 1900.0])
        }
        self.FY_INIT_POS = {
            'FY1': np.array([17800.0, 0.0, 1800.0]),
            'FY2': np.array([12000.0, 1400.0, 1400.0]),
            'FY3': np.array([6000.0, -3000.0, 700.0]),
            'FY4': np.array([11000.0, 2000.0, 1800.0]),
            'FY5': np.array([13000.0, -2000.0, 1300.0])
        }
        self.M_VECTORS = {k: (self.FAKE_TARGET_POS - v) / np.linalg.norm(v -
            self.FAKE_TARGET_POS) for k, v in self.M_INIT_POS.items()}
        self.M_FLIGHT_TIMES = {k: np.linalg.norm(v - self.FAKE_TARGET_POS) /
            self.MISSILE_V for k, v in self.M_INIT_POS.items()}
        self.ANALYSIS_STEP = 0.0001 # 步长

    def generate_points(self, num_circles=3, num_point=8):
        points = []
        for h in np.linspace(0, self.TARGET_H, num_circles):
            for i in range(num_point):
                angle = 2 * np.pi * i / num_point
                p = self.TARGET_BASE_POS + np.array([
                    self.TARGET_R * np.cos(angle),
                    self.TARGET_R * np.sin(angle),
                    h
                ])
                points.append(p)
            return np.array(points)

    @jit(nopython=True)
    def isInsmoke(p1, p2, s_pos, s_r):
        line_vec = p2 - p1
        line_len_sq = np.dot(line_vec, line_vec)
        if line_len_sq < 1e-9:
            return False, 0

```



```

p_vec = s_pos - p1
t = np.dot(p_vec, line_vec) / line_len_sq
if t < 0.0 or t > 1.0:
    return False, 0
dist_sq = np.dot(p_vec, p_vec) - t * t * line_len_sq
is_shielded = dist_sq < s_r * s_r
return is_shielded, 1 if is_shielded else 0

@jit(nopython=True)
def isShielded(m_pos, smokes_pos, target_points, s_r):
    num_points = target_points.shape[0]
    num_smokes = smokes_pos.shape[0]
    total_count = 0
    for i in range(num_points):
        target_point = target_points[i]
        smokes_lineNum = 0
        for j in range(num_smokes):
            smoke_pos = smokes_pos[j]
            is_blocked, count = isInSmoke(m_pos, target_point, smoke_pos, s_r)
            if is_blocked:
                smokes_lineNum += count
        if smokes_lineNum == 0:
            return False, 0.0
        if smokes_lineNum > 1:
            total_count += (smokes_lineNum - 1)
    overlap_contribution = total_count / num_points
    return True, overlap_contribution

def calculate_shield(
    detonation_pos,
    detonation_times,
    m_init_pos,
    m_vectors,
    m_flight_times,
    target_points,
    missile_v,
    cloud_r,
    smoke_sink_v,
    analysis_step,
    shield_duration
):
    num_missiles = m_init_pos.shape[0]

    max_flight_time = 0.0
    for i in range(num_missiles):
        if m_flight_times[i] > max_flight_time:
            max_flight_time = m_flight_times[i]

    num_steps = int(np.ceil(max_flight_time / analysis_step))
    time_steps = np.linspace(0, max_flight_time, num_steps)

    shielded_time_matrix = np.zeros((num_steps, num_missiles))
    overlap_time_matrix = np.zeros((num_steps, num_missiles))

    for t_idx in prange(num_steps):
        t = time_steps[t_idx]

```

```

active_pos_list = []
for i in range(detonation_pos.shape[0]):
    if t >= detonation_times[i] and t <= detonation_times[i] + shield_duration:
        s_pos = detonation_pos[i] + smoke_sink_v * (t - detonation_times[i])
        active_pos_list.append(s_pos)

    if len(active_pos_list) == 0:
        continue

active_pos = np.array(active_pos_list)

for m_idx in range(num_missiles):
    if t > m_flight_times[m_idx]:
        continue

m_pos = m_init_pos[m_idx] + m_vectors[m_idx] * missile_v * t

is_shielded, overlap_contrib = isShielded(
    m_pos,
    active_pos,
    target_points,
    cloud_r
)

if is_shielded:
    shielded_time_matrix[t_idx, m_idx] = 1

overlap_time_matrix[t_idx, m_idx] = overlap_contrib

return shielded_time_matrix, overlap_time_matrix

class Q5Result:
    def __init__(self, params):
        self.params = np.array(params).reshape(5, 8)
        self.config = Config()
        self.uav_ids = list(self.config.FY_INIT_POS.keys())
        self.missile_ids = list(self.config.M_INIT_POS.keys())

    def analyze(self):

        detonation_pos = np.zeros((15, 3))
        detonation_times = np.zeros(15)
        drop_pos = np.zeros((15, 3))

        smoke_idx = 0
        for i, uav_id in enumerate(self.uav_ids):
            v_fy, alpha_deg, t_drop1, t_delay1, delta_t2, t_delay2, delta_t3, t_delay3 =
                self.params[i]

            fy_init_pos = self.config.FY_INIT_POS[uav_id]
            alpha_rad = math.radians(alpha_deg)
            fy_vector = np.array([np.cos(alpha_rad), np.sin(alpha_rad), 0.0])
            v_at_drop = fy_vector * v_fy

            t_drops = np.array([t_drop1, t_drop1 + delta_t2, t_drop1 + delta_t2 + delta_t3])
            t_delays = np.array([t_delay1, t_delay2, t_delay3])

```

```

for j in range(3):
    t_drop = t_drops[j]
    t_delay = t_delays[j]

    current_drop_pos = fy_init_pos + fy_vector * v_fy * t_drop
    current_detonation_pos = current_drop_pos + v_at_drop * t_delay + 0.5 *
        np.array([0.0, 0.0, -self.config.G]) * t_delay**2
    t_detonation = t_drop + t_delay

    drop_pos[smoke_idx] = current_drop_pos
    detonation_pos[smoke_idx] = current_detonation_pos
    detonation_times[smoke_idx] = t_detonation
    smoke_idx += 1

m_init_pos_arr = np.array([self.config.M_INIT_POS[mid] for mid in
    self.missile_ids])
m_vectors_arr = np.array([self.config.M_VECTORS[mid] for mid in
    self.missile_ids])
m_flight_times_arr = np.array([self.config.M_FLIGHT_TIMES[mid] for mid in
    self.missile_ids])

shielded_time_matrix, overlap_time_matrix = calculate_shield(
    detonation_pos, detonation_times,
    m_init_pos_arr, m_vectors_arr, m_flight_times_arr,
    self.config.target_feature_points, self.config.MISSILE_V, self.config.CLOUD_R,
    self.config.SMOKE_SINK_V, self.config.ANALYSIS_STEP, self.config.SHIELD_DURATION
)

total_shield_time_per_missile = np.sum(shielded_time_matrix, axis=0) *
    self.config.ANALYSIS_STEP
total_shield_time_all_missiles = np.sum(total_shield_time_per_missile)
total_overlap_time = np.sum(overlap_time_matrix) * self.config.ANALYSIS_STEP

results = []

smoke_contribution_times = np.zeros((len(self.missile_ids), 15))

num_steps = shielded_time_matrix.shape[0]
for t_idx in range(num_steps):
    for m_idx in range(len(self.missile_ids)):

        if shielded_time_matrix[t_idx, m_idx]:
            m_pos = m_init_pos_arr[m_idx] + m_vectors_arr[m_idx] * self.config.MISSILE_V *
                (t_idx * self.config.ANALYSIS_STEP)

            active_smoke_indices = []
            t = t_idx * self.config.ANALYSIS_STEP
            for s_idx in range(15):
                if t >= detonation_times[s_idx] and t <= detonation_times[s_idx] +
                    self.config.SHIELD_DURATION:
                    active_smoke_indices.append(s_idx)

            for fp in self.config.target_feature_points:
                blocking_smokes_for_fp = set()
                for s_idx in active_smoke_indices:
                    s_pos = detonation_pos[s_idx] + self.config.SMOKE_SINK_V * (t -
                        detonation_times[s_idx])
                    is_blocked, _ = isInsmoke(m_pos, fp, s_pos, self.config.CLOUD_R)

```

```

if is_blocked:
    blocking_smokes_for_fp.add(s_idx)

if len(blocking_smokes_for_fp) > 0:
    contribution = self.config.ANALYSIS_STEP / len(self.config.target_feature_points)
    for s_idx in blocking_smokes_for_fp:
        smoke_contribution_times[m_idx, s_idx] += contribution

smoke_idx = 0
for i, uav_id in enumerate(self.uav_ids):
    v_fy, alpha_deg, _, _, _, _ = self.params[i]
    for j in range(3): # 3弹

shield_times_per_missile = smoke_contribution_times[:, smoke_idx]

if np.sum(shield_times_per_missile) > 1e-4:
    target_missile_idx = np.argmax(shield_times_per_missile)
    target_missile_id = self.missile_ids[target_missile_idx]
    effective_shield_time = np.sum(shield_times_per_missile) # 总贡献时长
else:
    target_missile_id = "无"
    effective_shield_time = 0.0

results.append({
    "编号": uav_id,
    "方向": f"{alpha_deg:.2f}",
    "速度 (m/s)": f"{v_fy:.2f}",
    "烟弹编号": j + 1,
    "烟弹投放点的x (m)": f"{drop_pos[smoke_idx, 0]:.2f}",
    "烟弹投放点的y (m)": f"{drop_pos[smoke_idx, 1]:.2f}",
    "烟弹投放点的z (m)": f"{drop_pos[smoke_idx, 2]:.2f}",
    "烟弹起爆点的x (m)": f"{detonation_pos[smoke_idx, 0]:.2f}",
    "烟弹起爆点的y (m)": f"{detonation_pos[smoke_idx, 1]:.2f}",
    "烟弹起爆点的z (m)": f"{detonation_pos[smoke_idx, 2]:.2f}",
    "有效干扰时长 (s)": f"{effective_shield_time:.4f}",
    "干扰的导弹编号": target_missile_id
})
smoke_idx += 1

df = pd.DataFrame(results)

print("\n"+" 最终结果 ")
print(f"\n 各导弹被遮蔽时间:")
for i, missile_id in enumerate(self.missile_ids):
    print(f" {missile_id}: {total_shield_time_per_missile[i]:.4f} s")
print(f"三个导弹被遮蔽时间之和: {total_shield_time_all_missiles:.4f} s")
total_smoke_contribution = np.sum(smoke_contribution_times)
print(f"烟弹总贡献时间: {total_smoke_contribution:.4f} s")
print(f"总重叠遮蔽时间: {total_overlap_time:.4f} s\n")

print("最终结果\n")
print(df.to_string())

def main():
    analyzer = Q5Result(OPTIMAL_PARAMS)
    analyzer.analyze()

```

```
if __name__ == '__main__':  
    main()
```